

Curs 1 - Principiile de baza ale securitatii

Amenintari si obiectivele securitatii

- Triada C-I-A (confidentiality, integrity, availability)
- Confidentialitate – bunurile protejate pot fi vazute doar de persoanele autorizate
- Integritate - bunurile protejate pot fi modificate doar de persoanele autorizate
- Disponibilitate (availability) - bunurile protejate pot fi utilizate doar de persoanele autorizate

Principiile securitatii

- Criptografice

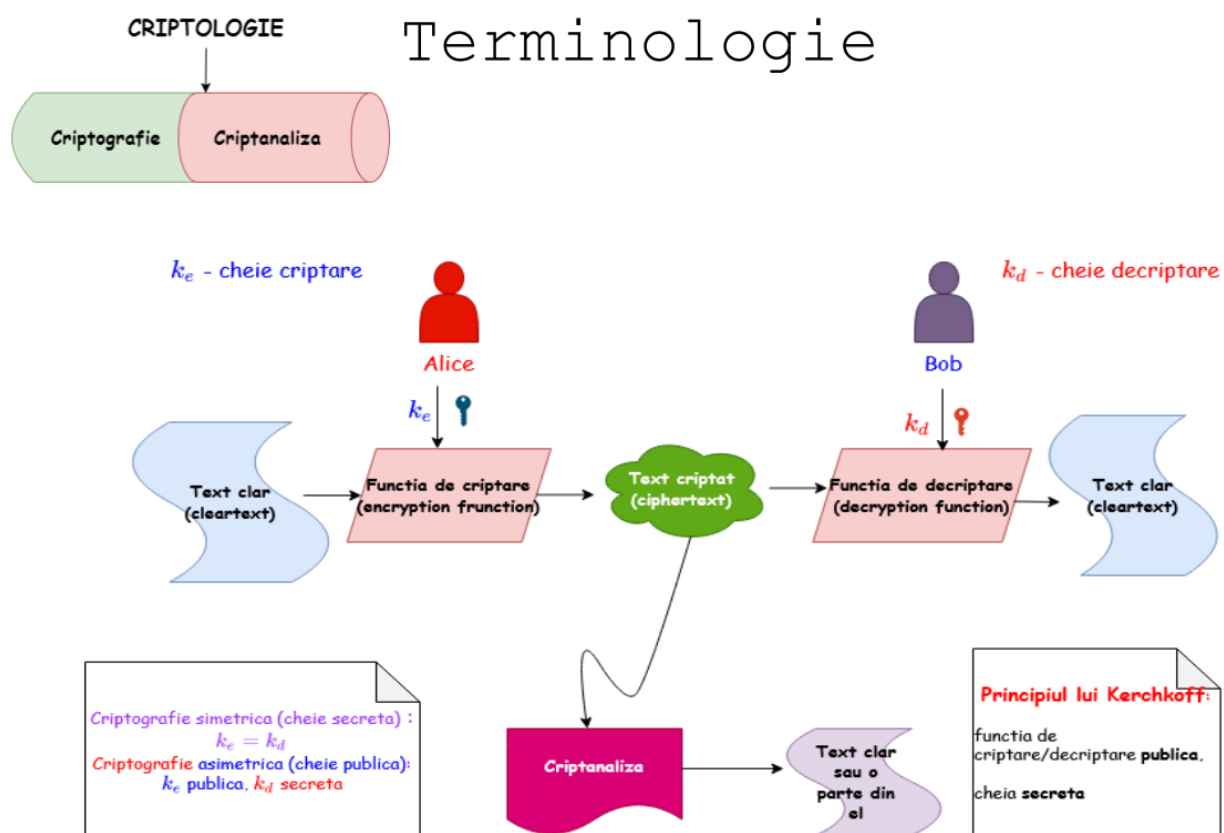
Principiul lui Kerkoff: doar cheia este secreta, constructia (algoritmul) e publica

Principiul separarii cheilor: chei diferite pentru scopuri diferite

Principiul diversitatii: foloseste tipuri diferite de algoritmi

Principiile securitatii

- Alte principii
- Principiul simplitatii: keep it simple
- Securitate implicita (security by default): trebuie gandita de la inceput, iar nu adaugata mai tarziu
- Principiul increderii minime (principle of minimal trust): minimizarea numarului de entitati carora le acordam incredere
- Principiul celei mai slabe verigi (principle of the weakest link): securitatea unui sistem este data de punctul sau cel mai slab
- Principiul celui mai mic privilegiu (least privilege): se acorda exact privilegiul necesar pentru efectuarea unei activitati
- Principiul modularitatii: totul trebuie pastrat modular
- Defence in depth – securitate la diverse nivele



- Criptografie: cum folosim k_e si k_d ca sa asiguram securitatea comunicatiilor pe un canal nesigur
- Securitate: cum protejeaza calculatorul/sistemul cheilestocate k_e si k_d de diverse atacuri (virusi, viermi etc.)

Ce proprietati de securitate asigura criptografia?

- Confidentialitatea (mesajelor) - adversarul nu vede sau nu poate obtine mesajul M

Text criptat: MARTEJIASCPT

Cifrul lui cezar:

a	b	c	d	e	f	g	h	i	j	k	l	m
D	E	F	G	H	I	J	K	L	M	N	O	P

n	o	p	q	r	s	t	u	v	w	x	y	z
Q	R	S	T	U	V	W	X	Y	Z	A	B	C

Text clar: mesaj criptat

Text criptat: PHVDM FULSWDW

- ▶ $|\mathcal{K}| = 26$
- ▶ **atac prin forță brută (căutare exhaustivă):** încercarea, pe rând, a tuturor cheilor posibile până când se obține un text clar cu sens

Principiul cheilor suficiente: O schemă sigură de criptare trebuie să aibă un spațiu al cheilor suficient de mare a.î. să nu fie vulnerabilă la căutarea exhaustivă.

Substitutia simpla:

a	b	c	d	e	f	g	h	i	j	k	l	m
F	I	L	O	R	U	X	A	D	G	J	M	P

n	o	p	q	r	s	t	u	v	w	x	y	z
S	V	Y	B	E	H	K	N	Q	T	W	Z	C

Text clar: mesaj criptat

Text criptat: PRHFG LEDYKFK

- ▶ $|\mathcal{K}| = 26!$
- ▶ atacul prin forță brută devine mai dificil
- ▶ **analiza de frecvență:** determinare corespondenței între alfabetul clar și alfabetul criptat pe baza frecvenței de apariție a literelor în text, cunoscând distribuția literelor în limba textului clar
 - ▶ se cunoaște limba textului clar
 - ▶ lungimea textului permite analiza de frecvență

Criptografia moderna

- Se bazează pe 3 principii moderne

Principiul 1 – orice problema criptografică necesită o definiție clasică și riguroasă – discutat în Curs 1 (scheme construite după acest principiu sunt folosite azi în TLS, SSH, IPSec)

Principiul 2 - securitatea primitivelor criptografice se bazează pe presupunții clare de securitate

Principiul 3 – orice construcție criptografică trebuie să fie însoțită de o demonstrație de securitate conform principiilor anterioare

Principiul 2 – presupunții(ipoteze) de securitate

- majoritatea construcțiilor criptografice moderne nu pot fi demonstrate ca fiind sigure necondiționat
- ipotezele de securitate trebuie să fie explicite:
 - permit cercetătorilor să valideze aceste ipoteze
 - permit comparația între două scheme bazate pe ipoteze diferite de securitate
 - implicații practice în cazul unor erori apărute în cadrul ipotezei de securitate
 - necesare pentru demonstrațiile de securitate

Exemplu de ipoteză de securitate (problema dificilă)

- Factorizarea numerelor mari

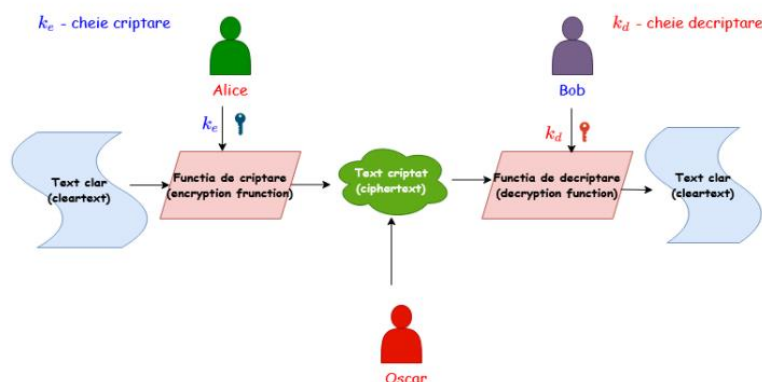
o Se dă un număr compus N și se cere descompunerea lui în factori primi. Ex: $85 = 17 * 5$

o Astăzi nu se cunoaște nici un algoritm care să factorizeze un număr de 400 cifre într-un timp practic

- Totuși:

1. Un algoritm mai rapid ar putea exista
2. Un calculator cuantic factorizează rapid (încă nu a fost construit dar se fac eforturi în acest sens)
3. Criptografia “post-cuantică” este în plină dezvoltare –competiția de standardizare post-cuantică NIST, criptografia bazată pe latici etc.

Securitate perfectă (Shannon 1949)



Ipoteză: Oscar cunoaște distribuția peste M

Securitate perfectă:

1. dacă Oscar află textul criptat nu are nici un fel de informație în plus decât dacă nu l-ar fi aflat (nu schimbă ceea ce știe el despre distribuția peste M).
- (textul criptat nu oferă nici un fel de informație despre textul clar)
2. Oscar nu poate ghici care din două posibile mesaje clare a fost criptat, doar văzând textul criptat

Definiție

O schemă de criptare peste un spațiu al mesajelor M este perfect sigură dacă pentru orice probabilitate de distribuție peste M , pentru orice mesaj $m \in M$ și orice text criptat c pentru care $Pr[C = c] > 0$, următoarea egalitate este îndeplinită:

$$Pr[M = m | C = c] = Pr[M = m]$$

- $Pr[M = m]$ - probabilitatea *a priori* ca Alice să aleagă mesajul m ;
- $Pr[M = m | C = c]$ - probabilitatea *a posteriori* ca Alice să aleagă mesajul m , chiar dacă textul criptat c a fost văzut ;
- **securitate perfectă** - dacă Oscar află textul criptat nu are nici un fel de informație în plus decât dacă nu l-ar fi aflat.

Definitie

O schemă de criptare (Enc, Dec) este perfect sigură dacă pentru orice mesaje $m_0, m_1 \in \mathcal{M}$ cu $|m_0| = |m_1|$ și $\forall c \in \mathcal{C}$ următoarea egalitate este îndeplinită:

$$Pr[Enc_k(m_0) = c] = Pr[Enc_k(m_1) = c]$$

unde $k \in \mathcal{K}$ este o cheie aleasă uniform.

- fiind dat un text criptat, este imposibil de ghicit dacă textul clar este m_0 sau m_1
- cel mai puternic adversar nu poate deduce nimic despre textul clar dat fiind textul criptat

One Time Pad (OTP)

mesaj clar:	0	1	1	0	0	1	1	1	1	⊕
cheie:	1	0	1	1	0	0	1	1	0	
text criptat:	1	1	0	1	0	1	0	0	1	

- **avantaj** - criptare și decriptare rapide
- **dezavantaj** - cheia foarte lungă (la fel de lungă precum textul clar)

mesaj clar:	0	1	1	0	0	1	1	1	1	⊕
cheie:	1	0	1	1	0	0	1	1	0	
text criptat:	1	1	0	1	0	1	0	0	1	

mesaj clar:	1	1	0	0	0	0	1	1	0	⊕
cheie:	0	0	0	1	0	1	1	1	1	
text criptat:	1	1	0	1	0	1	0	0	1	

- Același text criptat poate să provină din orice text clar cu o cheie potrivită
- Dacă adversarul nu știe decât textul criptat, atunci nu știe nimic despre textul clar!
- Securitatea perfectă nu este imposibilă dar..
- ❗ cheia trebuie să fie la fel de lungă precum mesajul
- ❗ inconveniente practice (stocare, transmisie)
- ❗ cheia trebuie să fie folosită o singură dată – one time pad
- Exercițiu: Ce se întâmplă dacă folosim o aceeași cheie de două ori cu sistemul OTP ?
- Dacă un adversar obține + - xor
- $C = M + K$ și $C' = M' + K$
- atunci el poate calcula
- $C + C' = (M + K) + (M' + K) = (M + M')$
- ceea ce invalidează proprietatea de securitate perfectă

Limitările securității perfecte

Teoremă

Fie o schemă (Enc, Dec) de criptare perfect sigură peste un spațiu al mesajelor $\mathcal{M}$ și un spațiu al cheilor $\mathcal{K}$. Atunci $|\mathcal{K}| \geq |\mathcal{M}|$.

Sau altfel spus

Teoremă

Nu există nici o schemă de criptare (Enc, Dec) perfect sigură în care mesajele au lungimea n biți iar cheile au lungimea (cel mult) $n - 1$ biți.

Curs 3-Securitate computacionala si aleatorism

Schema de criptare OTP este perfect sigura.

Majoritatea constructiilor criptografice moderne $\rightarrow$ securitate computacionala;

Schemele moderne pot fi sparte daca un atacator are la dispozitie suficient spatiu si putere de calcul.

Securitate perfecta vs. Criptografie computacionala

- Securitatea computacionala mai slaba decat securitatea informational-teoretica;
- Prima se bazeaza pe prezumptii de securitate; a doua este neconditionata;
- Intrebare: de ce renuntam la securitatea perfecta?
- Raspuns: datorita limitarilor practice!
- Preferam un compromis de securitate pentru a obtine constructii practice

Securitate computacionala

- Ideea de baza: principiul 1 al lui Kerckhoffs

Un cifru trebuie sa fie practic, daca nu matematic, indescifrabil.

- Sunt de interes mai mare schemele care practic nu pot fi sparte desi nu beneficiaza de securitate perfecta;

1. Adversari limitati computacional/eficienti/timp polinomial

Exemplu: Un atacator care realizeaza un atac prin forta bruta peste spatiul cheilor si testeaza o cheie/ciclu de ceas

- calculator desktop - se pot testa aprox. 257 chei/an
- supercalculator - se pot testa aprox. 280 chei/an
- supercalculator, varsta universului - 2112 chei

2. Adversarii pot efectua un atac cu succes cu o probabilitate foarte mica;

Exemplu: un adversar afla textul clar cu probabilitate 2^{-60} într-un an

Indistinctibilitate perfecta

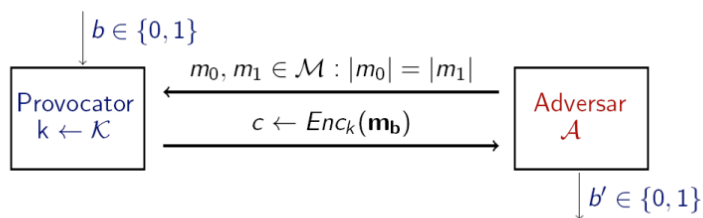
- Pentru securitatea perfecta am dat doua definitii echivalente, a doua sublinia ideea de indistinctibilitate: adversarul nu poate distinge intre criptarile a doua mesaje diferite
- Vom defini indistinctibilitatea pe baza unui experiment

$$Priv_{A,\pi}^{eav}$$

unde $\pi = (Enc, Dec)$ este schema de criptare

- Personaje participante: adversarul A care incearca sa sparga schema si un provocator (challenger).
- Trebuie sa definim capabilitatile adversarului: el poate vedea un singur text criptat cu o anume cheie, fiind un adversary pasiv care poate rula atacuri in timp polinomial, si nu are nici o alta interactiune cu Alice sau Bob

Experimentul $Priv_{\mathcal{A},\pi}^{eav}$



- Output-ul experimentului este 1 dacă $b' = b$ și 0 altfel. Dacă $Priv_{\mathcal{A},\pi}^{eav} = 1$, spunem că $\mathcal{A}$ a efectuat experimentul cu succes.
- Schema π este *perfect indistinctibilă* dacă

$$Pr[Priv_{\mathcal{A},\pi}^{eav}(n) = 1] = \frac{1}{2}$$

- Reamintim ca *indistinctibilitatea perfectă* este doar o definiție alternativă pentru *securitatea perfectă*

O schemă de criptare este (t, ϵ) -indistinctibilă dacă orice adversar care rulează în timp cel mult t

$$Pr[Priv_{\mathcal{A},\pi}^{eav} = 1] \leq \frac{1}{2} + \epsilon$$

- probabilitatea de succes a adversarului $\leq \epsilon$
- adversarul rulează în timp $\leq t$
- dezavantaj: am dori să avem scheme în care utilizatorul își poate ajusta nivelul de securitate dorit

Neglijabil și ne-neglijabil

- *în practică*: ϵ este scalar și
 - ϵ ne-neglijabil dacă $\epsilon \geq 1/2^{30}$
 - ϵ neglijabil dacă $\epsilon \leq 1/2^{80}$
- *în teorie*: ϵ este funcție $\epsilon : \mathbb{Z}_{\geq 0} \rightarrow \mathbb{R}_{\geq 0}$ și $p(n)$ este o funcție polinomială în n (ex.: $p(n) = n^d$, d constantă)
 - ϵ ne-neglijabilă în n dacă $\exists p(n) : \epsilon(n) > 1/p(n)$
 - ϵ neglijabilă în n dacă $\forall p(n), \exists n_d$ a.î. $\forall n \geq n_d : \epsilon(n) \leq 1/p(n)$

Definiție

Un *sistem de criptare simetric* definit peste $(\mathcal{K}, \mathcal{M}, \mathcal{C})$, cu:

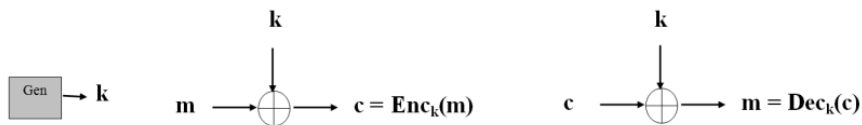
- $\mathcal{K}$ = spațiul cheilor
- $\mathcal{M}$ = spațiul textelor clare (mesaje)
- $\mathcal{C}$ = spațiul textelor criptate

este un triplet $(\text{Gen}, \text{Enc}, \text{Dec})$, unde:

1. $\text{Gen}(1^n)$: este algoritmul probabilistic de generare a cheilor care întoarce o cheie k conform unei distribuții
2. Enc : primește o cheie k și un mesaj $m \in \{0, 1\}^*$ și întoarce $c \leftarrow \text{Enc}_k(m)$
3. Dec : primește cheia k și textul criptat și întoarce m sau "eroare".

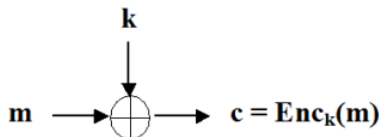
Aleatorism

- ▶ Incercăm criptare în stil OTP: cheia va masca mesajul dar
 - ▶ masca nu va fi doar cheia ci masca = $f(\text{cheie})$ unde f este o functie de extindere a cheii
 - ▶ pentru securitate perfecta masca trebuie sa fie perfect aleatoare
 - ▶ pentru securitate computațională, este suficient ca masca sa para aleatoare pentru un adversar PPT chiar daca nu este
- ▶ Vom avea nevoie întâi să definim noțiunea de *generatoare de numere pseudoaleatoare* ca element important de construcție pentru schemele de criptare simetrice

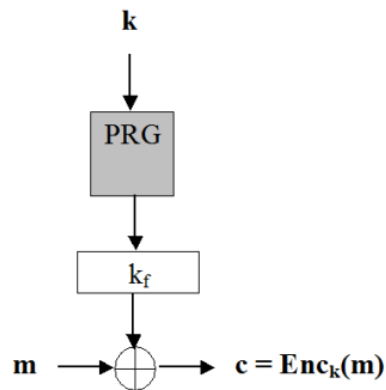


pseudoaleatorismul este o relaxare a aleatorismului perfect așa cum securitatea computațională este o relaxare a securității perfecte

OTP (One Time Pad)



Sistem de cripare



PRG (PseudoRandom Generator);

- ▶ Acesta este un algoritm determinist care primește o samantă relativ scurtă s (seed) și generează o secvență pseudoaleatoare de biți;
- ▶ Notăm $|s| = n$, $|\text{PRG}(s)| = l(n)$
- ▶ PRG prezintă interes dacă: $l(n) \geq n$ (altfel NU "generează aleatorism")

Notatii

- ▶ D = Distinguisher
- ▶ PPT = Probabilistic Polynomial Time
- ▶ $x \leftarrow_R X$ = x este ales uniform aleator din X
- ▶ $\text{negl}(n)$ = o funcție neglijabilă în (parametrul de securitate) n

În plus:

- ▶ Vom nota A un adversar (Oscar / Eve), care (în general) are putere polinomială de calcul

- ▶ Considerăm următorul PRG: $G(s) = s || \bigoplus_{i=1}^n s_i$
- ▶ factorul de expansiune $l(n) = n + 1$
- ▶ Considerăm algoritmul D astfel: $D(w) = 1$ dacă și numai dacă ultimul bit al lui w este egal cu xor-ul tuturor biților precedenți
- ▶ Se verifica ușor ca $Pr[D(G(s)) = 1] = 1$
- ▶ Dacă r este uniform, atunci bitul final al lui r este uniform și deci $Pr[D(r) = 1] = \frac{1}{2}$
- ▶ $|\frac{1}{2} - 1|$ nu e neglijabilă și deci G nu este PRG

Observații

- ▶ Distribuția output-ului unui PRG este departe de a fi uniformă
- ▶ Exemplificăm pentru un G care dublează lungimea intrării i.e. $l(n) = 2n$
- ▶ Pentru distribuția uniformă peste $\{0, 1\}^{2n}$, fiecare din cele 2^{2n} este ales cu probabilitate ...
- ▶ ... $\frac{1}{2^{2n}}$
- ▶ Considerăm distribuția output-ului lui G când primește la intrare un sir uniform de lungime n
- ▶ Numarul de siruri diferite din codomeniul lui G este cel mult ...
- ▶ ... 2^n
- ▶ Probabilitatea ca un sir de lungime $2n$ să fie output al lui G este $2^n / 2^{2n} = 1/2^n$

Obs:

- ▶ Seed-ul unui PRG este analogul cheii unui sistem de criptare
- ▶ seed-ul trebuie ales uniform și menținut secret
- ▶ seed-ul trebuie să fie suficient de lung așa încât un atac prin forță brută să nu fie fezabil

Definiție

Un sistem de criptare (Enc, Dec) definit peste $(\mathcal{K}, \mathcal{M}, \mathcal{C})$ se numește **sistem de criptare bazat pe PRG** dacă:

1. $Enc : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$

$$c = Enc_k(m) = G(k) \oplus m$$

2. $Dec : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$

$$m = Dec_k(c) = G(k) \oplus c$$

unde G este un generator de numere pseudoaleatoare cu factorul de expansiune l , $k \in \{0, 1\}^n$, $m \in \{0, 1\}^{l(n)}$

- ▶ OTP este perfect sigur;

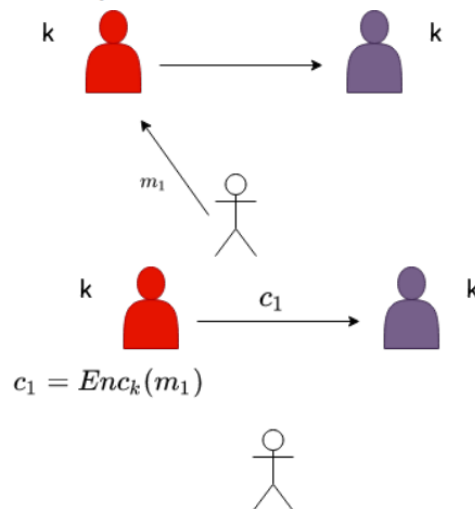
- Criptarea bazata pe PRG se obtine din OTP prin inlocuirea pad cu $G(k)$;
- Daca G este PRG, atunci pad si $G(k)$ sunt indistinctibile pentru orice A adversar PPT;
- In concluzie, OTP si sistemul de criptare bazat pe PRG sunt indistinctibile pentru A.

Curs 4: Notiuni de securitate mai puternice, Sisteme fluide - stream ciphers, Sisteme de criptare bloc

- Reamintim cateva dintre scenariile de atac pe care le-am mai intalnit:
- Atac cu text criptat: Atacatorul stie doar textul criptat – poate incerca un atac prin forta bruta prin care se parcurg toate cheile pana se gaseste cea corecta;
- Atac cu text clar: Atacatorul cunoaste una sau mai multe perechi (text clar, text criptat);
- Atac cu text clar ales: Atacatorul poate obtine criptarea unor texte clare alese de el;
- Atac cu text criptat ales: Atacatorul are posibilitatea sa obtina decriptarea unor texte criptate alese de el.

Securitate CPA

- CPA (Chosen-Plaintext Attack): adversarul poate să obțină criptarea unor mesaje alese de el;



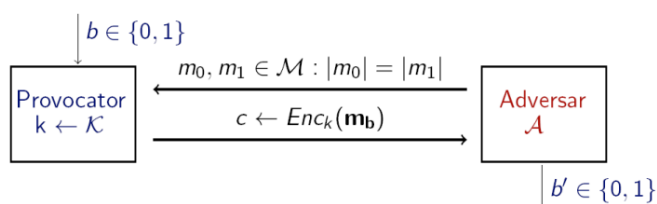
adversarul poate cere criptarea unor mesaje alese de el repetitiv (polinomial de multe ori)

mai tarziu adversarul observa criptarea unui mesaj necunoscut

dorim ca adversarul sa nu afle nici un fel de informatie despre mesajul m

- Capabilitatile adversarului: el poate interactiona cu un oracol de criptare, fiind un adversar activ care poate rula atacuri in timp polinomial;
- Adversarul poate transmite catre oracol orice mesaj m si primeste inapoi textul criptat corespunzator;
- Daca sistemul de criptare este nedeterminist, atunci oracolul foloseste de fiecare data o valoare aleatoare noua, si neutilizata anterior.

Experimentul $\text{Priv}_{\mathcal{A}, \pi}^{\text{cpa}}(n)$



- Pe toată durata experimentului, $\mathcal{A}$ are acces la oracolul de criptare $\text{Enc}_k(\cdot)$!

Definiție

O schemă de criptare $\pi = (Enc, Dec)$ este **CPA-sigură** dacă pentru orice adversar PPT $\mathcal{A}$ există o funcție neglijabilă $negl$ așa încât

$$Pr[Priv_{\mathcal{A},\pi}^{cpa}(n) = 1] \leq \frac{1}{2} + negl(n).$$

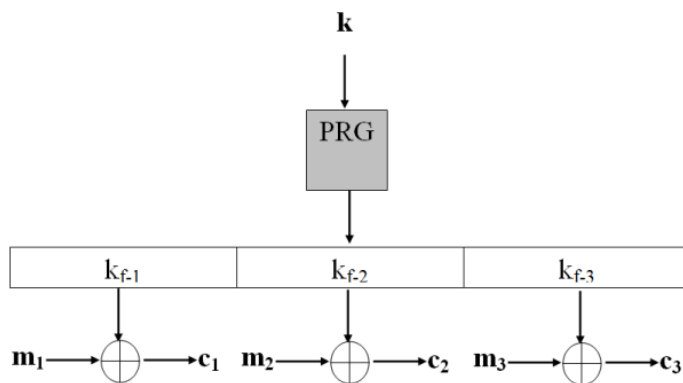
- ▶ Un adversar nu poate determina care text clar a fost criptat cu o probabilitate semnificativ mai mare decât dacă ar fi ghicit (în sens aleator, dat cu banul), chiar dacă are acces la oracolul de criptare.
- ▶ **Întrebare:** Un sistem de criptare CPA-sigur are întotdeauna proprietatea de indistinctibilitate?
- ▶ **Răspuns:** DA! Experimentul $Priv_{\mathcal{A},\pi}^{eav}(n)$ este $Priv_{\mathcal{A},\pi}^{cpa}(n)$ în care $\mathcal{A}$ nu folosește oracolul de criptare.
- ▶ **Întrebare:** Un sistem de criptare determinist poate fi CPA-sigur?
- ▶ **Răspuns:** NU! Adversarul cere oracolului criptarea mesajului m_0 . Dacă textul criptat este egal cu c , atunci $b' = 0$, altfel $b' = 1$. În concluzie, $\mathcal{A}$ câștigă cu probabilitate 1.

Sisteme fluide

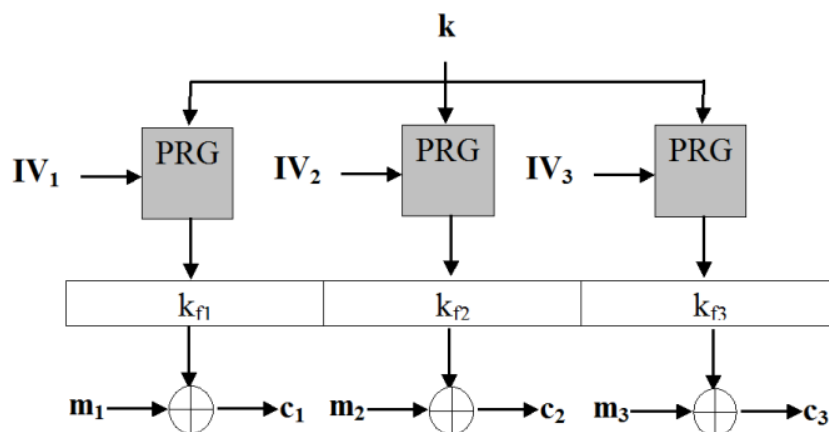
- ▶ sistemele fluide produc biții de output (pseudo-aleatori) gradual și la cerere, fiind mai **eficiente** și **flexibile**
- ▶ criptarea cu un sistem fluid presupune 2 faze:
 - ▶ **Faza 1:** se generează o secvență pseudoaleatoare de biți, folosind un **generator de numere pseudoaleatoare (PRG)**
 - ▶ **Faza 2:** secvența obținută se XOR-ează cu mesajul clar
- ▶ **Atenție!** De multe ori când ne referim la un sistem de criptare fluid considerăm doar Faza 1

Securitate - interceptare multiplă

- ▶ Un sistem de criptare fluid în varianta prezentată este determinist: unui text clar îi corespunde întotdeauna același mesaj criptat;
- ▶ În consecință, utilizarea unui sistem fluid în forma prezentată pentru criptarea mai multor mesaje (cu aceeași cheie) este nesigură;
- ▶ Un sistem de criptare fluid se folosește în practică în 2 moduri: sincronizat și nes sincronizat.
- ▶ **modul sincronizat:** partenerii de comunicație folosesc pentru criptarea mesajelor părți succesive ale secvenței pseudoaleatoare generate;



► **modul nesincronizat:** partenerii de comunicare folosesc pentru criptarea mesajelor secvențe pseudoaleatoare diferite.



Modul sincronizat

- mesajele sunt criptate în mod **succesiv** (participanții trebuie să știe care părți au fost deja folosite)
- necesită **păstrarea** stării
- mesajele succesive pot fi percepute ca un *singur mesaj clar lung*, obținut prin concatenarea mesajelor succesive
- se pretează unei singure sesiuni de comunicații

Modul nesincronizat

- mesajele sunt criptate în mod **independent**
- NU necesită **păstrarea** stării
- valorile $IV_1, IV_2, \dots$ sunt alese uniform aleator pentru fiecare mesaj transmis
- valorile $IV_1, IV_2, \dots$ (dar și IV în modul sincronizat) fac parte din mesajul criptat (sunt necesare pentru decriptare)

Fie $G(s, IV)$ un PRG cu 2 intrări:

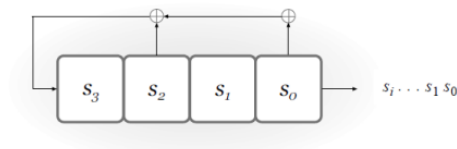
► s = seed

► IV = Initialization Vector

PRG trebuie să se satisfacă (cel puțin):

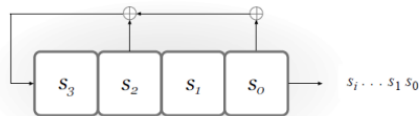
1. $G(s, IV)$ este o secvență pseudoaleatoare chiar dacă IV este public (i.e. securitatea lui G constă în securitatea lui s);
2. dacă IV_1 și IV_2 sunt valori uniform aleatoare, atunci $G(s, IV_1)$ și $G(s, IV_2)$ sunt indistinguibile

Linear-Feedback Shift Registers (LFSR)



În exemplul de mai sus avem

- ▶ $c_0 = c_2 = 1$ și $c_1 = c_3 = 0$
- ▶ fiecare bit de la ieșire este calculat după formula $c_0 s_0 \oplus \dots \oplus c_3 s_3$
- ▶ la fiecare tact de ceas, LFSR scoate la ieșire valoarea din registrul s_0 iar valorile din ceilalți regiștri sunt deplasate la dreapta cu o poziție



Pentru starea inițială (0,0,1,1), biții de la ieșire vor fi ...

(0,0,1,1)

(1,0,0,1)

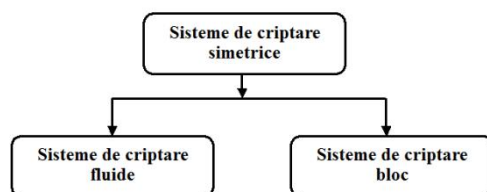
...

În general

- ▶ starea LFSR constă din n biți (conținutul regiștrilor la un moment dat)
- ▶ există cel mult 2^n stări posibile până când output-ul LFSR-ului se repetă

Vulnerabilitati LFSR ▶ LFSR-urile sunt liniare iar liniaritatea induce vulnerabilitati (sistemele liniare de ecuații permit aflarea informațiilor sensibile) ▶ Însă combinațiile de mai multe LFSR-uri pot produce sisteme de criptare sigure

- ▶ Am studiat sisteme simetrice care criptează **bit cu bit** - **sisteme de criptare fluide**;
- ▶ Vom studia sisteme simetrice care criptează **câte n biți simultan** - **sisteme de criptare bloc**;



... d.p.d.v. al modului de criptare:

Sisteme fluide

- ▶ criptarea biților se realizează **individual**
- ▶ criptarea unui bit din textul clar este **independentă** de orice alt bit din textul clar

Sisteme bloc

- ▶ criptarea se realizează în **blocuri** de câte n biți
- ▶ criptarea unui bit din textul clar este **dependentă** de biții din textul clar care aparțin aceluiași bloc

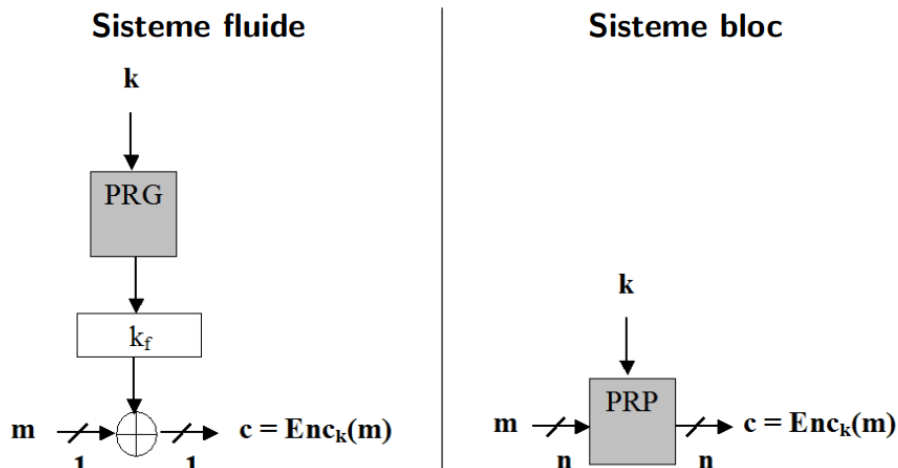
Sisteme fluide

- necesități computaționale reduse
- utilizare: telefoane mobile, dispozitive încorporate, PDA
- par să fie mai puțin sigure, multe sunt sparte

Sisteme bloc

- necesități computaționale mai avansate
- utilizare: internet
- par să fie mai sigure, prezintă încredere mai mare

► Introducem notiunea de permutare pseudoaleatoare sau PRP(PseudoRandom Permutation) ► In analogie cu ce stim deja: ► PRP sunt necesare pentru constructia sistemelor bloc asa cum ► PRG sunt necesare pentru constructia sistemelor fluide



PRP (PseudoRandom Permutation); ► Acesta este o functie **determinista** si **bijectiva** care pentru o cheie fixata produce la iesire o **permutare** a intrarii ... ► ... **indistinctibila** fata de o permutare aleatoare; ► In plus, atat functia cat ,si inversa sa sunt **eficient calculabile**.

functie pseudoaleatoare sau **PRF** (PseudoRandom Function)... ► ... ca o generalizare a notiunii de permutare pseudoaleatoare; ► Acesta este o functie cu cheie care este indistinctibila fata de o functie aleatoare (cu acelasi domeniu si multime de valori).

$$PRP \subseteq PRF$$

► **Întrebare:** De ce *PRF* poate fi privită ca o generalizare a *PRP*?

► **Răspuns:** *PRP* este *PRF* care satisface:

1. $\mathcal{X} = \mathcal{Y}$
2. este inversabilă
3. calculul funcției inverse este eficient

► $PRF \Rightarrow PRG$

Pornind de la PRF se poate construi PRG

► $PRG \Rightarrow PRF$

Pornind de la PRG se poate construi PRF

► $\text{PRP} \Rightarrow \text{PRF}$

Pornind de la PRP se poate construi PRF

► $\text{PRF} \Rightarrow \text{PRP}$

Pornind de la PRF se poate construi PRP

Intrebare: Care dintre aceste constructii este triviala?

$\text{PRP} \Rightarrow \text{PRF}$

► Intrebare: Ce se intampla daca lungimea mesajului clar este mai mica decat dimensiunea unui bloc?

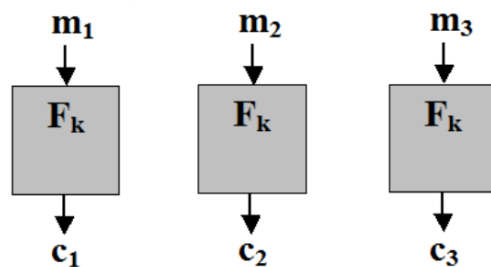
► Raspuns: Se completeaza cu biti: $1\ 0\ \dots\ 0$;

► Intrebare: Ce se intampla daca lungimea mesajului clar este mai mare decat lungimea unui bloc?

► Raspuns: Se utilizeaza un mod de operare (ECB, CBC, OFB, CTR);

► Notam cu F_k un sistem de criptare bloc (i.e. PRP) cu cheia k fixata.

ECB (Electronic Code Book)



► Pare modul cel mai natural de a cripta mai multe blocuri;

► Pentru decriptare, F_k trebuie sa fie inversabila;

► Este paralelizabil;

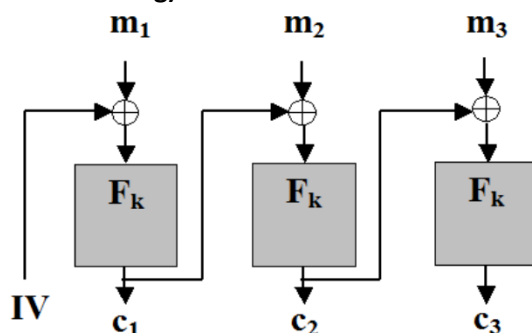
► Este determinist, deci este nesigur;

► Intrebare: Ce informatii poate sa ofere modul de criptare ECB unui adversar pasiv?

► Raspuns: Un adversar pasiv detecteaza repetarea unui bloc de text clar pentru ca se repeta blocul criptat corespunzator;

► Modul ECB NU trebuie utilizat in practica!

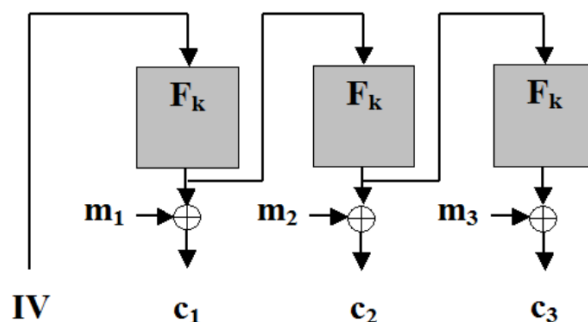
CBC (Cipher Block Chaining)



► IV este o ales in mod aleator la criptare; ► IV se transmite in clar pentru ca este necesar la decriptare;

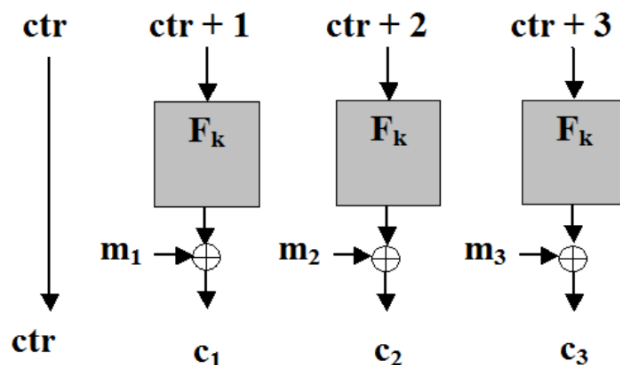
► Pentru decriptare, F_k trebuie sa fie inversabila; ► Este secvential, un dezavantaj major daca se poate utiliza procesarea paralela.

OFB (Output FeedBack)



► Generează o secvență pseudoaleatoare care se XOR-ează mesajului clar; ► IV este ales în mod aleator la criptare; ► IV se transmite în clar pentru ca este necesar la decriptare; ► F_k nu trebuie neapărat să fie inversabilă; ► Este secvențial, însă secvența pseudoaleatoare poate fi pre-procesată anterior decriptării.

CTR (Counter)



► Generează o secvență pseudoaleatoare care se XOR-ează mesajului clar; ► ctr este ales în mod aleator la criptare; ► ctr se transmite în clar pentru ca este necesar la decriptare; ► F_k nu trebuie neapărat să fie inversabilă; ► Este paralelizabil; ► În plus, secvența pseudoaleatoare poate fi pre-procesată anterior decriptării.

- Generează o secvență pseudoaleatoare care se XOR-ează mesajului clar;
- ctr este ales în mod aleator la criptare și se transmite în clar pentru ca este necesar la decriptare;
- Este **paralelizabil**; F_k nu trebuie neapărat să fie inversabilă;
- În plus, secvența pseudoaleatoare poate fi pre-procesată anterior decriptării.
- CTR poate fi văzut și ca un sistem fluid nesincronizat:
 - pentru criptarea unui mesaj de lungime $l < 2^{n/4}$ blocuri, se alege un IV uniform din $\{0, 1\}^{2^{n/4}}$
 - fiecare bloc de text criptat este calculat $y_i = F_k(IV || i)$ unde i este codificat ca un string pe $n/4$ biți

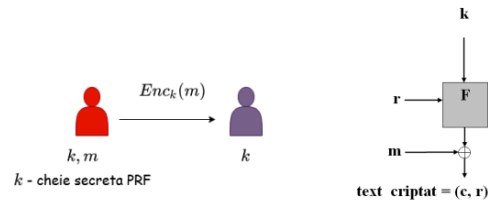
!!!!

- modurile CTR, OFB și CBC sunt CPA-sigure
- modurile CBC, OFB și CTR folosesc un IV uniform aleator - asigură faptul că F_k este mereu evaluat pe intrări diferite (previne situația în care adversarul afla informații la vederea de intrări identice)
- CTR - IV ales uniform de lungime $3n/4$ înseamnă că IV se repetă după criptarea aprox. $q(n) = 2^{2n/8}$ mesaje
- Dacă $n = 64$ atunci $q \approx 17.000.000$ ceea ce e puțin pentru zilele noastre
- Dacă $n = 128$ și vrem să folosim CTR având garanția că IV se repetă cu probabilitate cel mult 2^{-40} , rezultă $q \approx 2^{28}$ mesaje (calculând q din $\frac{q^2}{2^{3n/4} + 1} \leq 2^{-40}$)

► Ce se întâmplă dacă IV se repetă? ► Pentru modurile OFB și CTR, întregul stream pseudoaleator (cu care se face xor pe mesaj) se repetă ► Dacă IV nu este uniform aleator (deci este predictibil), CTR este sigur dar CBC nu este sigur.

Curs 5 - Scheme de criptare CPA-sigure bazate pe PRF, Securitate CCA, Construcții practice PRF

- Sistemele de criptare bloc sunt instanțieri sigure ale PRP



- Fie F_k o funcție cu cheie
- $\text{Gen}(1^n)$: alege uniform cheie $k \in \{0, 1\}^n$
- $\text{Enc}_k(m)$: pentru $|m| = |k|$, alege r uniform în $\{0, 1\}^n$

$$\text{Enc}_k(m) = (r, F_k(r) \oplus m)$$

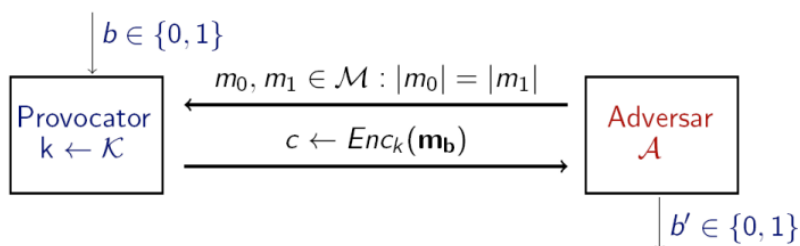
- $\text{Dec}_k(c = (c_0, c_1))$: întoarce $c_1 \oplus F_k(c_0)$

Securitatea Sistemelor Informatic

3 / 7

CCA

- **CPA (Chosen-Plaintext Attack)**: adversarul poate să obțină criptarea unor mesaje alese de el; - discutată în cursul anterior
- **CCA (Chosen-Ciphertext Attack)**: adversarul poate să obțină criptarea unor mesaje alese de el și decriptarea unor texte criptate alese de el.
 - Capabilitățile adversarului: el poate interacționa cu un oracol de criptare și cu un oracol de decriptare, fiind un adversary activ care poate rula atacuri în timp polinomial;
 - Adversarul poate transmite către oracolul de criptare orice mesaj m și primește înapoi textul criptat corespunzător sau poate transmite către oracolul de decriptare anumite mesaje c și primește înapoi mesajul clar corespunzător;
 - Dacă sistemul de criptare este nedeterminist, atunci oracolul de criptare folosește de fiecare dată o valoare aleatoare nouă și neutilizată anterior



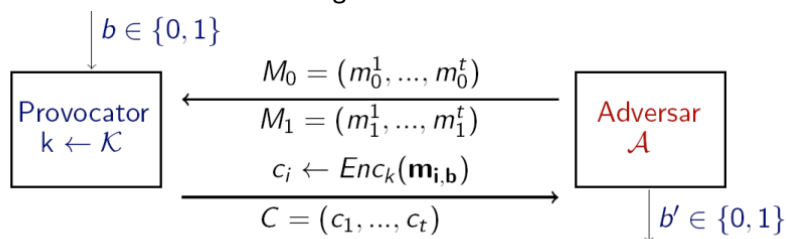
- Output-ul experimentului este 1 dacă $b' = b$ și 0 altfel. Dacă $\text{Priv}_{\mathcal{A}, \pi}^{\text{cca}}(n) = 1$, spunem că $\mathcal{A}$ a efectuat experimentul cu succes.

Definiție

O schemă de criptare $\pi = (Enc, Dec)$ este **CCA-sigură** dacă pentru orice adversar PPT $\mathcal{A}$ există o funcție neglijabilă $negl$ așa încât

$$Pr[Priv_{\mathcal{A}, \pi}^{cca}(n) = 1] \leq \frac{1}{2} + negl(n).$$

- Un adversar nu poate determina care text clar a fost criptat cu o probabilitate semnificativ mai mare decât dacă ar fi ghicit (în sens aleator, dat cu banul), chiar dacă are acces la oracolele de criptare și decriptare.
- Intrebare: Un sistem de criptare CCA-sigur este întotdeauna CPA-sigur?
- Raspuns: DA! Experimentul $Priv_{cpa}$
- $A, \pi(n)$ este $Priv_{cca}$ $A, \pi(n)$ în care A nu folosește oracolul de decriptare.
- Intrebare: Un sistem de criptare determinist poate fi CCA-sigur?
- Raspuns: NU! Sistemul nu este CPA-sigur, deci nu poate fi CCA-sigur.
- În definiția precedentă am considerat cazul unui adversar care primește un singur text criptat; ► În realitate, în cadrul unei comunicatii se trimit mai multe mesaje pe care adversarul le poate intercepta; ► Definim ce înseamnă o schemă sigură chiar și în aceste condiții



- Pe toată durata experimentului, $\mathcal{A}$ are acces la oracolul de criptare $Enc_k(\cdot)$ și la oracolul de decriptare $Dec_k(\cdot)$ cu restricția că nu poate decripta $c_1, \dots, c_t$!

Nici una din schemele de criptare de până acum nu sunt CCA-sigure.

Schemele deterministe nu sunt semantic / CPA / CCA sigure

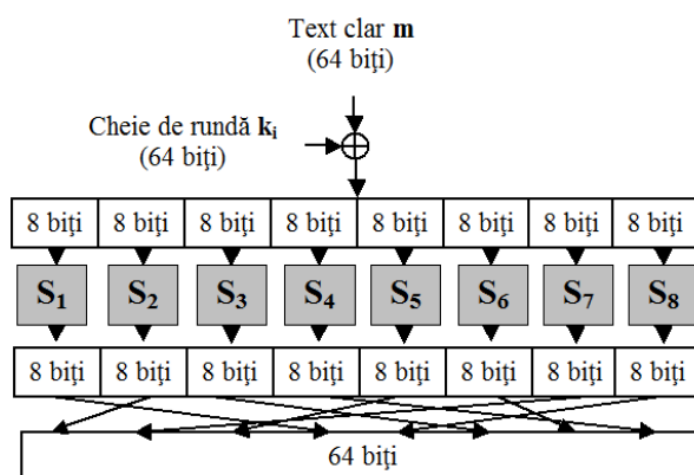
- Se construiește funcția F , pe baza mai multor funcții aleatoare f_i de dimensiune mai mică;
- Considerăm F pe 128 biți și 16 funcții aleatoare $f_1, \dots, f_{16}$ pe câte 8 biți;
- Pentru $x = x_1 || \dots || x_{16}$, $x \in \{0, 1\}^{128}$ $x_i \in \{0, 1\}^8$:

$$F_k(x) = f_1(x_1) || \dots || f_{16}(x_{16})$$

- Spunem că $\{f_i\}$ introduc **confuzie** în F .

Rețele de substituție - permutare F încă nu este PRF dar ► F se transformă în PRF în 2 pași: ► Pasul 1: se introduce difuzie prin amestecarea (permutarea) bitilor de ieșire; ► Pasul 2: se repetă o rundă (care presupune confuzie și difuzie) de mai multe ori; ► Repetarea confuziei și difuziei face ca modificarea unui singur bit de intrare să fie propagată asupra tuturor bitilor de ieșire;

- O rețea de **substituție-permutare** este o implementare a construcției anterioare de *confuzie-difuzie* în care funcțiile $\{f_i\}$ sunt **fixe** (i.e. nu depind de cheie) și se numesc permutări;
- $\{f_i\}$ se numesc **S-boxes** (Substitution-boxes);
- Cum nu mai depind de cheie, aceasta este utilizată în alt scop;
- Din cheie se obțin mai multe **chei de rundă** (*sub-chei*) în urma unui proces de derivare a cheilor (*key schedule*);
- Fiecare cheie de rundă este XOR-ată cu valorile intermediare din fiecare rundă.



- Există 2 principii de bază în proiectarea rețelelor de substituție - permutare:
 - **Principiul 1:** Inversabilitatea S-box-urilor;
 - dacă toate S-box-urile sunt inversabile, atunci rețeaua este inversabilă;
 - necesitate funcțională (pentru decriptare)
 - **Principiul 2:** Efectul de avalanșă
 - Un singur bit modificat la intrare **trebuie** să afecteze toți biții din secvența de ieșire;
 - necesitate de securitate.

AES

- AES este o rețea de substituție - permutare pe 128 biți care poate folosi chei de 128, 192 sau 256 biți;
- Lungimea cheii determină numărul de runde:

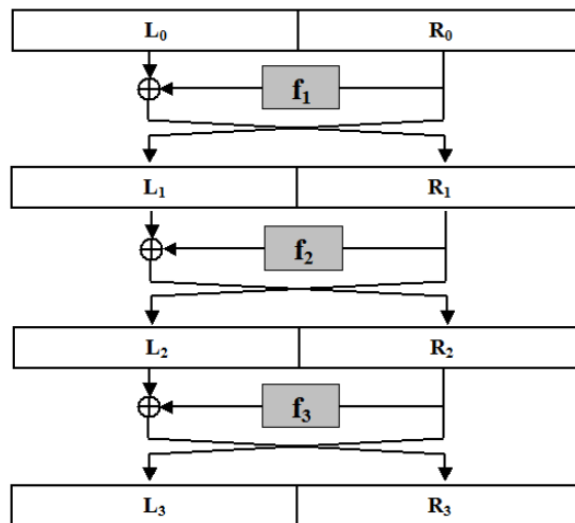
Lungime cheie (biți)	128	192	256
Număr runde	10	12	14

- Folosește o matrice de octeți 4×4 numită **stare**;
- Starea inițială este mesajul clar ($4 \times 4 \times 8 = 128$);
- Starea este modificată pe parcursul rundelor prin 4 tipuri de operații: *AddRoundKey*, *SubBytes*, *ShiftRows*, *MixColumns*;
- Ieșirea din ultima rundă este textul criptat.

- ▶ Singurele atacuri netriviale sunt asupra AES cu număr redus de runde:
 - ▶ AES-128 cu 6 runde: necesită 2^{72} criptări;
 - ▶ AES-192 cu 8 runde: necesită 2^{188} criptări;
 - ▶ AES-256 cu 8 runde: necesită 2^{204} criptări.
- ▶ Nu există un atac mai eficient decât căutarea exhaustivă pentru AES cu număr complet de runde.

RETELE FEISTEL

- ▶ Se aseamănă rețelelor de substituție-permutare în sensul că păstrează aceleași elementele componente: S-box, permutare, procesul de derivare a cheii, runde;
- ▶ Se diferențiază de rețelele de substituție-permutare prin proiectarea de nivel înalt;
- ▶ Introduc avantajul major că S-box-urile NU trebuie să fie inversabile;
- ▶ Permit așadar obținerea unei structuri *inversabile* folosind elemente *neinversabile*.



- ▶ Intrarea în runda i se împarte în 2 jumătăți: L_{i-1} și R_{i-1} (i.e. *Left* și *Right*);
- ▶ Ieșirile din runda i sunt:

$$L_i = R_{i-1}$$

$$R_i = L_{i-1} \oplus f_i(R_{i-1})$$

- ▶ Funcțiile f_i depind de cheia de rundă, derivând dintr-o funcție publică $\hat{f}_i$:

$$f_i(R) = \hat{f}_i(k_i, R)$$

- Rețelele Feistel sunt inversabile indiferent dacă funcțiile f_i sunt inversabile sau nu;
- Fie (L_i, R_i) ieșirile din runda i ;
- Intrările (L_{i-1}, R_{i-1}) în runda i sunt:

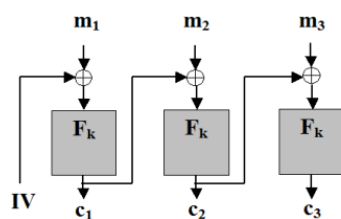
$$R_{i-1} = L_i$$

$$L_{i-1} = R_i \oplus f_i(R_{i-1})$$

Curs 6 - Padding-oracle attack, Constructii practice PRF, Data Encryption Standard – DES

Atacul bazat pe oracol de padding

- Am văzut cum funcționează modul CBC când lungimea mesajului clar este multiplu de lungimea L blocului de criptat (suportat de F_k) în octeți



Decriptare

$$m_1 = F_k^{-1}(c_1) \oplus IV$$

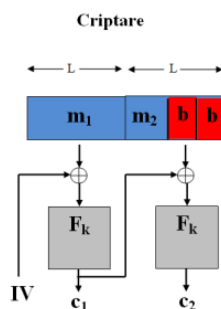
$$m_2 = F_k^{-1}(c_2) \oplus c_1$$

$$m_3 = F_k^{-1}(c_3) \oplus c_2$$

- Ce se întâmplă când $|m| \neq L$?
- Folosim padding-ul *PKCS#7* :
 - Fie b numărul de octeți de adăugat la ultimul bloc pentru a avea $|m|$ multiplu de L ($1 \leq b \leq L$)
 - Se adaugă b octeți la ultimul bloc din m , fiecare reprezentând valoarea lui b

CBC cu padding PKCS7

Considerăm situația în care un client trimite mesaje criptate în modul CBC către un server.



- în urma decriptării se obțin $m_1 || m_2$
- se citește octetul final b
- dacă ultimii b octeți au toți valoarea b , atunci se scoate padding-ul și se obține mesajul original m
- altfel întoarce mesajul *padding gresit* și cere retransmiterea mesajului

Server-ul acționează ca un oracol de padding - adversarul îi trimite texte criptate și află dacă padding-ul este corect sau nu

Ideea atacului cu oracol de padding

- Unui text criptat (IV, c) îi corespunde textul clar cu padding
 $m' = F_k^{-1}(c) \oplus IV$
- Dacă un adversar modifică octetul i din IV atunci modificarea se va reflecta și în octetul i din m'

$F_k^{-1}(c)$

XX	XX	XX	XX	XX	XX	XX	XX
----	----	----	----	----	----	----	----

$\oplus$

IV

CD	02	A7	19	23	7B	00	E8
----	----	----	----	----	----	----	----

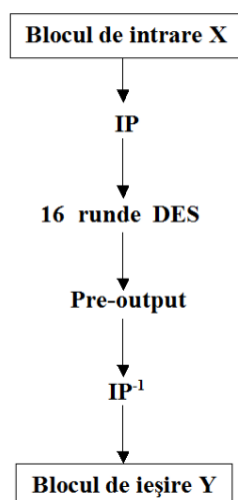
=

mesajul cu
padding

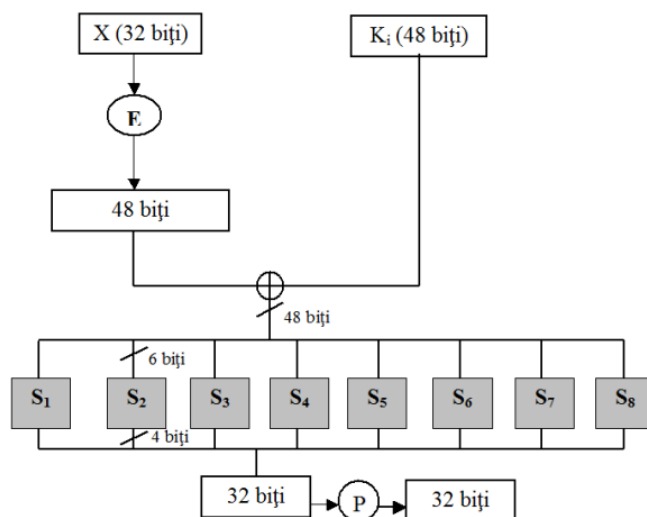
XX	XX	XX	XX	XX	XX	XX	XX
----	----	----	----	----	----	----	----

Complexitatea atacului cu oracol de padding ► sunt necesare cel mult L incercari pentru a afla b ► cel mult $2^8 = 256$ incercari pentru a afla fiecare octet din mesajul initial ► in total sunt necesare $256 * bt$ incercari (unde bt reprezinta numarul de octeti din mesajul original) pentru a gasi intregul text clar

DES



- DES este o rețea de tip Feistel cu 16 runde și o cheie pe 56 biți;
- Procesul de derivare a cheii (*key schedule*) obține o sub-cheie de rundă k_i pentru fiecare rundă pornind de la cheia master k ;
- Funcțiile de rundă $f_i(R) = f(k_i, R)$ sunt derivate din aceeași funcție principală $\hat{f}$ și nu sunt inversabile;
- Fiecare sub-cheie k_i reprezintă permutarea a 48 biți din cheia master;
- Întreaga procedură de obținere a sub-cheilor de rundă este fixă și cunoscută, singurul secret este cheia master .



- Funcția f este, în esență, o rețea de substituție-permutare cu doar o rundă.
- Pentru calculul $f(k_i, R)$ se procedează astfel:
 1. R este expandat la un bloc R' de 48 biți cu ajutorul unei funcții de expandare $R' = E(R)$.
 2. R' este XOR-at cu k_i iar valoarea rezultată este împărțită în 8 blocuri de câte 6 biți.
 3. Fiecare bloc de 6 biți trece printr-un SBOX diferit rezultând o valoare pe 4 biți.
 4. Se concatenează blocurile rezultate și se aplică o permutare, rezultând în final un bloc de 32 biți.
- **De remarcat:** Toată descrierea lui DES, inclusiv SBOX-urile și permutările sunt publice.

SBOX-urile din DES

- Formează o parte esențială din construcția DES;
- DES devine mult mai vulnerabil la atacuri dacă SBOX-urile sunt modificate ușor sau dacă sunt alese aleator
- Primul și ultimul bit din cei 6 de la intrare sunt folosiți pentru a alege linia din tabel, iar biții 2-5 sunt folosiți pentru coloană; output-ul va consta din cei 4 biți aflați la intersecția liniei și coloanei alese.

S_5		Cei 4 biți din mijloc															
		0000	0001	0010	0011	0100	0101	0110	0111	1000	1001	1010	1011	1100	1101	1110	1111
Biții din margine	00	0010	1100	0100	0001	0111	1010	1011	0110	1000	0101	0011	1111	1101	0000	1110	1001
	01	1110	1011	0010	1100	0100	0111	1101	0001	0101	0000	1111	1010	0011	1001	1000	0110
	10	0100	0010	0001	1011	1010	1101	0111	1000	1111	1001	1100	0101	0110	0011	0000	1110
	11	1011	1000	1100	0111	0001	1110	0010	1101	0110	1111	0000	1001	1010	0100	0101	0011

- Modificarea **unui bit** de la intrare întotdeauna afectează cel puțin **doi biți** de la ieșire.
- DES are un puternic efect de avalanșă generat de ultima

Securitatea sistemului DES

- ▶ Incă de la propunerea sa, DES a fost criticat din două motive:
 1. Spațiul cheilor este prea mic făcând algoritmul vulnerabil la forță brută;
 2. Criteriile de selecție a SBOX-urilor au fost ținute secrete și ar fi putut exista atacuri analitice care explorau proprietățile matematice ale SBOX-urilor, cunoscute numai celor care l-au proiectat.
- ▶ Cu toate acestea....
 - ▶ După 30 ani de studiu intens, cel mai bun atac practic rămâne doar o căutare exhaustivă pe spațiul cheilor;
 - ▶ O căutare printre 2^{56} chei este fezabilă azi (dar netrivială);
 - ▶ În 1977, un calculator care să efectueze atacul într-o zi ar fi costat 20.000.000\$;

▶ Primul atac practic a fost demonstrat în 1997 când un număr de provocări DES au fost propuse de RSA Security și rezolvate; ▶ Prima provocare a fost spartă în 1997 de un proiect care a folosit sute de calculatoare coordonate prin Internet; a durat 96 de zile; ▶ A doua provocare a fost spartă anul următor în 41 de zile; ▶ Impresionant a fost timpul pentru a treia provocare: 56 de ore; ▶ S-a construit o mașină în acest scop, Deep Crack cu un cost de 250.000\$

▶ O altă problemă a sistemului DES, mai puțin importantă, este lungimea blocului relativ scurtă (64 biți); ▶ Securitatea multor construcții bazate pe cifruri bloc depinde de lungimea blocului; ▶ În modul de utilizare CTR, dacă un atacator are 2^{27} perechi text clar/text criptat, securitatea este compromisă cu probabilitate mare; ▶ Concluzionând, putem spune că insecuritatea sistemului DES nu are a face cu structura internă sau construcția în sine (care este remarcabilă), ci se datorează numai lungimii cheii prea mici.

Criptanaliza avansată ▶ La sfârșitul anilor '80, Biham și Shamir au dezvoltat o tehnică numită criptanaliza diferențială pe care au folosit-o pentru un atac împotriva DES; ▶ Atacul presupune complexitate timp 2^{37} (memorie neglijabilă) dar cere ca atacatorul să analizeze 2^{36} texte criptate obținute dintr-o mulțime de 2^{47} texte clare alese; ▶ Din punct de vedere teoretic, atacul a fost o inovație, dar practic e aproape imposibil de realizat; ▶ La începutul anilor '90, Matsui a dezvoltat criptanaliza liniară aplicată cu succes pe DES; ▶ Deși necesită 2^{43} texte criptate, avantajul este că textele clare nu trebuie să fie alese de atacator, ci doar cunoscute de el; ▶ Problema însă rămâne aceeași: atacul e foarte greu de pus în practică

Criptanaliza diferențială

- ▶ Cataloghează diferențe specifice între texte clare care produc diferențe specifice în textele criptate cu probabilitate mai mare decât ar fi de așteptat pentru permutările aleatoare;
- ▶ Fie un cifru bloc cu blocul de lungime n și $\Delta_x, \Delta_y \in \{0, 1\}^n$;
- ▶ Spunem că diferențiala (Δ_x, Δ_y) apare cu probabilitate p dacă pentru intrări aleatoare x_1, x_2 cu

$$x_1 \oplus x_2 = \Delta_x$$

și o alegere aleatoare a cheii k

$$\Pr[F_k(x_1) \oplus F_k(x_2) = \Delta_y] = p$$

- ▶ Pentru o funcție aleatoare, probabilitatea de apariție a unei diferențiale nu e mai mare decât 2^{-n} ;
- ▶ La un cifru bloc slab, ea apare cu o probabilitate mult mai mare;

Criptanaliza liniară

- ▶ Metoda consideră relații liniare între intrările și ieșirile unui cifru bloc;
- ▶ Spunem că porțiunile de biți $i_1, \dots, i_l$ și $i'_1, \dots, i'_l$ au distanța p dacă pentru orice intrare aleatoare x și orice cheie k

$$\Pr[x_{i_1} \oplus \dots \oplus x_{i_l} \oplus y_{i'_1} \oplus \dots \oplus y_{i'_l} = 0] = p$$

unde $y = F_k(x)$.

- ▶ Pentru o funcție aleatoare se așteaptă ca $p = 0.5$;
- ▶ Matsui a arătat cum se poate folosi o diferență p mare pentru a sparge complet un cifru bloc;
- ▶ Necesită un număr foarte mare de perechi text clar/text criptat.

Criptare dublă

- ▶ Fie F un cifru bloc (în particular ne vom referi la DES); definim un alt cifru bloc F' astfel

$$F'_{k_1, k_2}(x) = F_{k_2}(F_{k_1}(x))$$

cu k_1, k_2 chei independente;

- ▶ Lungimea totală a cheii este 112 biți, suficient de mare pentru căutare exhaustivă;
- ▶ Însă, se poate arăta un atac în timp 2^{56} unde $|k_1| = 56 = |k_2|$ (fața de 2^{112} cât necesită o căutare exhaustivă);
- ▶ Atacul se numește **meet-in-the-middle**;

Atacul meet-in-the-middle

- ▶ Iată cum funcționează atacul dacă se cunoaște o pereche text clar/text criptat (x, y) cu $y = F_{k_2}(F_{k_1}(x))$:
 1. Pentru fiecare $k_1 \in \{0, 1\}^n$, calculează $z := F_{k_1}(x)$ și păstrează (z, k_1) ;
 2. Pentru fiecare $k_2 \in \{0, 1\}^n$, calculează $z := F_{k_2}^{-1}(y)$ și păstrează (z, k_2) ;
 3. Verifică dacă există perechi (z, k_1) și (z, k_2) care coincid pe prima componentă;
 4. Atunci valorile k_1, k_2 corespunzătoare satisfac

$$F_{k_1}(x) = F_{k_2}^{-1}(y)$$

adică $y = F'_{k_1, k_2}(x)$

- ▶ Complexitatea timp a atacului este $O(2^n)$.

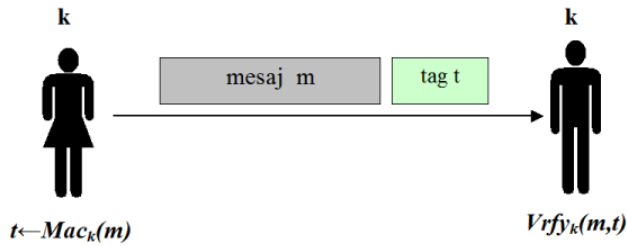
Triplu-DES (3DES) ► Se bazează pe tripla invocare a lui DES folosind două sau trei chei; ► Este considerat sigur și în 1999 l-a înlocuit pe DES ca standard; ► 3DES este foarte eficient în implementările hardware (la fel ca și DES) dar totuși lent în implementări software; ► 3DES cu 3 chei încă se mai folosește dar recomandarea este de a înlocui datorită lungimii mici a blocurilor și a faptului că este lent ► Este popular în aplicațiile financiare și în protejarea informațiilor biometrice din pașapoartele electronice;

- DES a fost sistemul simetric dominant de la mijlocul anilor '70 până la mijlocul anilor '90;
 - DES cu cheia pe 56 biți poate fi spart relativ ușor astăzi prin forță brută;
 - Însă, este foarte greu de spart folosind criptanaliza diferențială sau liniară;
 - Pentru 3DES nu se cunoaște nici un atac practic.
-

Curs 7- Coduri de autentificare a mesajelor – MAC, Funcții hash

Criptare vs. autentificarea mesajelor

- Criptarea, în general, **NU** oferă integritatea mesajelor!
 - Nu folosiți criptarea cu scopul de a obține autentificarea mesajelor
 - Dacă un mesaj este transmis criptat de-a lungul unui canal de comunicare, nu înseamnă că un adversar nu poate modifica/altera mesajul așa încât modificarea să aibă sens în textul clar;
 - Verificăm, în continuare, că nici o schemă de criptare studiată nu oferă integritatea mesajelor;
-
- Așa cum am văzut, criptarea nu rezolvă problema autentificării mesajelor;
 - Vom folosi un mecanism diferit, numit **cod de autentificare a mesajelor - MAC (Message Authentication Code)**;
 - Scopul lor este de a împiedica un adversar să modifice un mesaj trimis fără ca părțile care comunică să nu detecteze modificarea;
 - Vom lucra în continuare în contextul criptografiei cu cheie secretă unde părțile trebuie să prestabilească de comun acord o cheie secretă.
-



- ▶ Alice și Bob stabilesc o cheie secretă k pe care o partajează;
- ▶ Când Alice vrea să îi trimită un mesaj m lui Bob, calculează mai întâi un tag t (o etichetă) pe baza mesajului m și a cheii k și trimite perechea (m, t) ;

▶ Tag-ul este calculat folosind un algoritm de generare a tag-urilor numit Mac ; ▶ La primirea perechii (m, t) Bob verifică dacă tag-ul este valid (în raport cu cheia k) folosind un algoritm de verificare Vrfy ; ▶ În continuare prezentăm definiția formală a unui cod de autentificare a mesajelor

Definiție

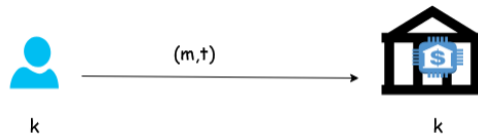
Un *cod de autentificare a mesajelor (MAC)* definit peste $(\mathcal{K}, \mathcal{M}, \mathcal{T})$ este format dintr-un triplet de algoritmi polinomiali $(\text{Gen}, \text{Mac}, \text{Vrfy})$ unde:

1. $\text{Gen}(1^n)$: este algoritmul de generare a cheii k (aleasă uniform pe n biți)
2. $\text{Mac} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{T}$ este algoritmul de generare a tag-urilor $t \leftarrow \text{Mac}_k(m)$;
3. $\text{Vrfy} : \mathcal{K} \times \mathcal{M} \times \mathcal{T} \rightarrow \{0, 1\}$ este algoritmul de verificare ce întoarce un bit $b = \text{Vrfy}_k(m, t)$ cu semnificația că:
 - ▶ $b = 1$ înseamnă valid
 - ▶ $b = 0$ înseamnă invalid

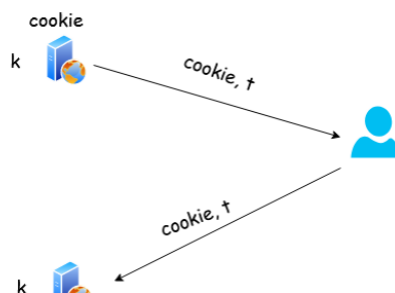
$$a.\hat{t} : \forall m \in \mathcal{M}, k \in \mathcal{K} \text{ Vrfy}_k(m, \text{Mac}_k(m)) = 1.$$

Cazuri de folosire ale MAC

1. când cele două părți care comunică partajează o cheie secretă în avans



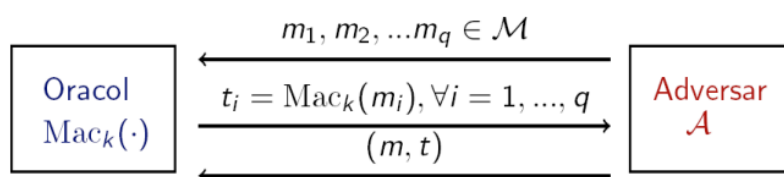
2. când avem o singură parte care autentifică o comunicație, interacționând cu ea însăși după un timp



Securitate MAC - discutie ► Intuitie: nici un adversar polinomial nu ar trebui sa poata genera un tag valid pentru nici un mesaj "nou" care nu a fost deja trimis (si autentificat) de partile care comunica; ► Trebuie sa definim puterea adversarului si ce inseamna spargerea sau un atac asupra securitatii; ► Adversarul lucreaza in timp polinomial si are acces la mesajele trimise intre parti impreuna cu tag-urile aferente. ► Adversarul este activ, poate influenta autentificarea unor mesaje alese de el... ► ...dar nu trebuie sa poata falsifica tag-ul aferent unor mesaje care nu au fost autentificate de expeditor

- Formal, îi dăm adversarului acces la un *oracol* $\text{Mac}_k(\cdot)$;
- Adversarul poate trimite orice mesaj m dorit către oracol și primește înapoi un tag corespunzător $t \leftarrow \text{Mac}_k(m)$;
- Considerăm că securitatea este impactată dacă adversarul este capabil să producă un mesaj m împreună cu un tag t așa încât:
 1. t este un tag valid pentru mesajul m : $\text{Vrfy}_k(m, t) = 1$;
 2. Adversarul nu a solicitat anterior (de la oracol) un tag pentru mesajul m .

- Despre un MAC care satisface nivelul de securitate de mai sus spunem că *nu poate fi falsificat printr-un atac cu mesaj ales*;
- Aceasta înseamnă că un adversar nu este capabil să falsifice un tag valid pentru nici un mesaj ...
- ... deși poate obține tag-uri pentru orice mesaj ales de el, chiar *adaptiv* în timpul atacului.
- Pentru a da definiția formală, definim mai întâi un experiment pentru un MAC $\pi = (\text{Mac}, \text{Vrfy})$, în care considerăm un adversar $\mathcal{A}$ și parametrul de securitate n ;



- Output-ul experimentului este 1 dacă și numai dacă:
 - (1) $\text{Vrfy}_k(m, t) = 1$ si
 - (2) $m \notin \{m_1, \dots, m_q\}$;
- Dacă $\text{Mac}_{\mathcal{A}, \pi}^{\text{forge}}(n) = 1$, spunem că $\mathcal{A}$ a efectuat experimentul cu succes.

► Intrebare: De ce este necesara a doua conditie de la securitatea MAC (un adversar nu poate intoarce un mesaj pentru care anterior a cerut un tag)? ► Raspuns: Pentru a evita atacurile triviale in care un adversar cere tag-ul aferent unui mesaj si apoi intoarce chiar acel mesaj impreuna cu tag-ul primit. ► Intrebare: Definitia MAC ofera protectie la atacurile prin replicare (in care un adversar copiaza un mesaj

împreună cu tag-ul aferent trimise de partile comunicante)? ► Raspuns: NU! MAC-urile nu ofera nici un fel de protectie la atacurile prin replicare.

MAC-uri sigure

- Avem nevoie de o funcție cu cheie, pentru care, dându-se $Mac_k(m_1), Mac_k(m_2), \dots$
- ... să nu fie ușor (în timp polinomial) a găsi $Mac_k(m)$ pentru orice $m \notin \{m_1, m_2, \dots\}$
- Funcția MAC ar putea fi un PRF

-
- *Funcțiile pseudoaleatoare (PRF) sunt un instrument bun pentru a construi MAC-uri sigure;*

Construcție

Fie $F : \{0, 1\}^n \times \{0, 1\}^n \rightarrow \{0, 1\}^n$ o PRF. Definim un MAC în felul următor:

- Mac : pentru o cheie $k \in \{0, 1\}^n$ și un mesaj $m \in \{0, 1\}^n$, calculează tag-ul $t = F_k(m)$ (dacă $|m| \neq |k|$ nu întoarce nimic);
- $Vrfy$: pentru o cheie $k \in \{0, 1\}^n$, un mesaj $m \in \{0, 1\}^n$ și un tag $t \in \{0, 1\}^n$, întoarce 1 dacă și numai dacă $t = F_k(m)$ (dacă $|m| \neq |k|$, întoarce 0).

Teorema Dacă F este PRF, construcția de mai sus reprezintă un cod de autentificare a mesajelor sigur (nu poate fi falsificat prin atacuri cu mesaj ales).

MAC-uri pentru mesaje de lungime variabilă

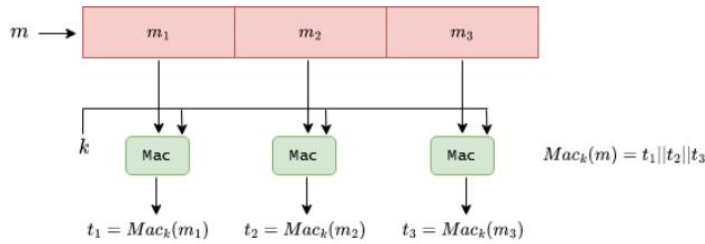
- Construcția prezentată anterior funcționează doar pe mesaje de lungime fixă (PRF-urile sunt instanțiate cu sisteme de criptare bloc care, cel mai adesea, acceptă input-uri de 128 biți);
- Însă în practică avem nevoie de mesaje de lungime variabilă;
- Arătăm cum putem obține un MAC de lungime variabilă pornind de la un MAC de lungime fixă;
- Fie $(\pi' = (Mac', Vrfy'))$ un MAC sigur de lungime fixă pentru mesaje de lungime n ;
- Pentru a construi un MAC de lungime variabilă, putem sparge mesajul m în blocuri $m_1, \dots, m_d$ și autentificăm blocurile folosind π' ;
- Iată câteva modalități de a face aceasta:

1. XOR pe toate blocurile cu autentificarea rezultatului:

$$t = Mac'_k(\oplus_i m_i)$$

- **Intrebare:** Este sigură această metodă?
 - **Răspuns:** NU! Un adversar poate modifica mesajul original m a.î. XOR-ul blocurilor nu se schimbă, el obținând un tag valid pentru un mesaj nou;
-

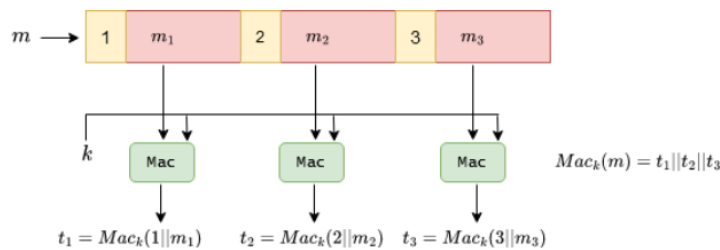
2. Autentificare separată pentru fiecare bloc:



- **Intrebare:** Este sigură această metodă?
- **Răspuns:** NU!
- Fiind dat (m, t) cu $m = m_1 || m_2 || m_3$ și $t = t_1 || t_2 || t_3$
- Atunci (m', t') este o pereche validă cu $m' = m_2 || m_1 || m_3$ și $t' = t_2 || t_1 || t_3$

3. Autentificare separată pentru fiecare bloc folosind o secvență de numere:

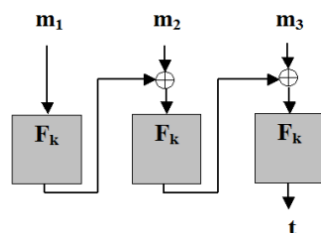
În felul acesta prevenim atacul anterior



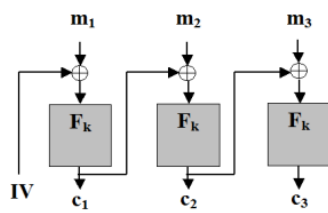
- **Intrebare:** Este sigură această metodă?
- **Răspuns:** NU! Un adversar poate scoate blocuri de la sfârșitul mesajului: $t = t_1 || t_2$ este un tag valid pentru mesajul $m = m_1 || m_2$;

CBC-MAC

- O soluție mult mai eficientă este să folosim **CBC-MAC**;
- CBC-MAC este o construcție similară cu modul CBC folosit pentru criptare;
- Folosind CBC-MAC, pentru un tag aferent unui mesaj de lungime $l \cdot n$, se aplică sistemul bloc doar de l ori, iar tag-ul are numai n biți.

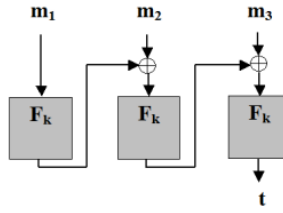


CBC-MAC vc. Criptare în modul CBC



Criptare în mod CBC

- ▶ IV este aleator pentru a obține securitate;
- ▶ toate blocurile c_i constituie mesajul criptat.



CBC-MAC

- ▶ $IV = 0^n$ este fixat pentru a obține securitate;
- ▶ doar ieșirea ultimului bloc constituie tag-ul (întoarcerea tuturor blocurilor intermediare duce la pierderea securității)

Confidențialitate și integritate - criptare autentificată

- ▶ Iată trei abordări uzuale pentru a combina criptarea și autentificarea mesajelor:

1. *Criptare-și-autentificare*: criptarea și autentificarea se fac independent. Pentru un mesaj clar m , se transmite mesajul criptat $\langle c, t \rangle$ unde

$$c \leftarrow \text{Enc}_{k_1}(m) \text{ și } t \leftarrow \text{Mac}_{k_2}(m)$$

La recepție, $m = \text{Dec}_{k_1}(c)$ și dacă $\text{Vrfy}_{k_2}(m, t) = 1$, atunci întoarce m ; altfel întoarce $\perp$.

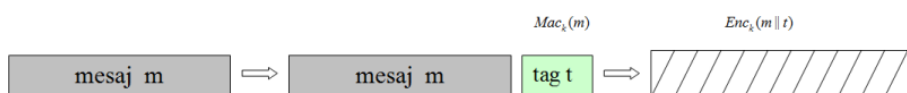


- ▶ Combinația aceasta **nu** este neapărat sigură; un MAC sigur nu implică nici un fel de confidențialitate;

2. *Autentificare-apoi-criptare*: întâi se calculează tag-ul t apoi mesajul și tag-ul sunt criptate împreună

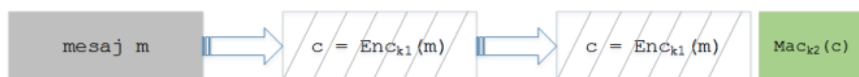
$$t \leftarrow \text{Mac}_{k_2}(m) \text{ și } c \leftarrow \text{Enc}_{k_1}(m || t)$$

La recepție, $m || t = \text{Dec}_{k_1}(c)$ și dacă $\text{Vrfy}_{k_2}(m, t) = 1$, atunci întoarce m ; altfel întoarce $\perp$.



- ▶ Combinația aceasta **nu** este neapărat sigură;
- ▶ Se poate construi o schemă de criptare CPA-sigură care împreună cu orice MAC sigur nu poate fi CCA-sigură;

3. Criptare-apoi-autentificare



- Combinația aceasta este întotdeauna sigură; se folosește în IPsec;
- Deși folosim aceeași construcție pentru a obține securitate CCA și transmitere sigură a mesajelor, scopurile urmărite sunt diferite în fiecare caz;

► Orice schema de criptare autentificată este și CCA-sigură dar există și scheme de criptare CCA-sigure care nu sunt scheme de criptare autentificate ► Totuși, cele mai multe aplicații de scheme de criptare cu cheie secretă în prezența unui adversar activ necesită integritate

CA în	bazată pe	care în general este	dar în acest caz este
SSH	C_si_A	nesigură	sigură
SSL	A_apoi_C	nesigură	nesigură
IPSec	C_apoi_A	sigură	sigură
WinZip	C_apoi_A	sigură	nesigură

- CA = criptare autentificată;
- C-apoi-A = criptare-apoi-autentificare;
- C-si-A = criptare-si-autentificare

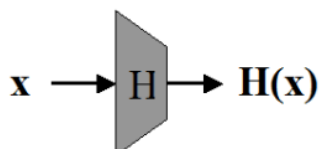
Funcții hash

Acestea primesc ca argument o secvență de lungime variabilă pe care o comprimă într-o secvență de lungime mai mică, fixată

Căutare $O(1)$

Întrebare: Ce se întâmplă dacă pentru 2 valori $x \neq x'$, $H(x) = H(x')$? ► Răspuns: În acest caz, apare o coliziune - ambele valori se vor stoca în aceeași celulă

- **Întrebare:** Există funcții hash fără coliziuni?
- **Răspuns:** NU! Funcțiile hash (cel puțin cele interesante d.p.d.v. criptografic) comprimă, deci funcția nu poate fi injectivă;



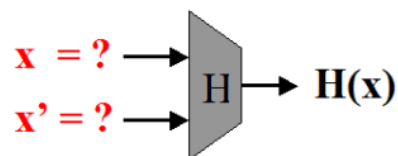
Definiție

$H : \{0, 1\}^* \rightarrow \{0, 1\}^{l(n)}$ se numește **rezistentă la coliziuni** (*collision-resistant*) dacă pentru orice adversar PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[\text{Hash}_{\mathcal{A}, H}^{\text{coll}}(n) = 1] \leq \text{negl}(n).$$

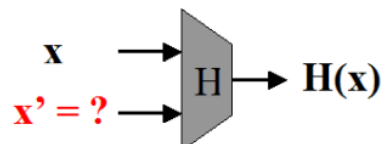
- În practică, rezistența la coliziuni poate fi dificil de obținut;
- Pentru anumite aplicații sunt utile noțiuni mai relaxate de securitate;
- Există 3 nivele de securitate:
 1. **Rezistența la coliziuni:** este cea mai puternică noțiune de securitate și deja am definit-o formal;
 2. **Rezistența la a doua preimagine:** presupune că fiind dat x este dificil de determinat $x' \neq x$ a.î. $H(x) = H(x')$
 3. **Rezistența la prima preimagine:** presupune că fiind dat $H(x)$ este imposibil de determinat x .

Coliziuni



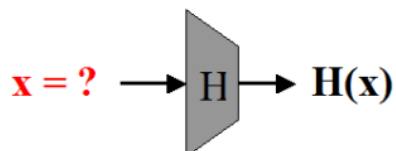
- **Provocare:** Se cer 2 valori $x \neq x'$ a.î. $H(x) = H(x')$;
- **Atac:** "Atacul zilei de naștere" necesită $\approx 2^{l(n)/2}$ evaluări pentru H .

A doua preimagine



- **Provocare:** Fiind dat x , se cere x' a.î. $H(x) = H(x')$;
- **Atac:** Un atac generic necesită $\approx 2^{l(n)}$ evaluări pentru H .

Prima preimagine



- **Provocare:** Fiind dat $y = H(x)$, se cere x a.î. $H(x) = y$;
- O astfel de funcție se numește și *calculabilă într-un singur sens (one-way function)*;
- **Atac:** Un atac generic necesită $\approx 2^{l(n)}$ evaluări pentru H .

- ▶ **Întrebare:** De ce o funcție care satisface proprietatea de **rezistență la coliziuni** satisface și proprietatea de **rezistență la a doua preimagine**?
- ▶ **Răspuns:** Pentru x fixat, adversarul determină $x' \neq x$ pentru care $H(x) = H(x')$, deci găsește o coliziune;
- ▶ **Întrebare:** De ce o funcție care satisface proprietatea de **rezistență la a doua preimagine** satisface și proprietatea de **rezistență la prima imagine**?
- ▶ **Răspuns:** Pentru x oarecare, adversarul calculează $H(x)$, o inversează și determină x' a.î. $H(x') = H(x)$. Cu probabilitate mare $x' \neq x$, deci găsește o a doua preimagine.

Atacul "zilei de naștere" ▶ Analizăm posibilitatea de a determina o coliziune pornind de la un exemplu clasic: paradoxul nasterilor; ▶ **Întrebare:** Care este dimensiunea unui grup de oameni pentru ca 2 dintre ei să fie născuți în aceeași zi cu probabilitate $1/2$? ▶ **Răspuns:** 23!

- ▶ Generalizând, considerăm o mulțime de dimensiune n și q elemente uniform aleatoare din această mulțime $y_1, \dots, y_q$;
- ▶ Atunci pentru $q \geq 1.2 \times 2^{n/2}$ probabilitatea să existe $i \neq j$ a.î. $y_i = y_j$ este $\geq 1/2$.
- ▶ Acest rezultat conduce imediat la un atac asupra funcțiilor hash cu scopul de a determina coliziuni:
 - ▶ Adversarul alege $2^{n/2}$ valori x_i ;
 - ▶ Calculează pentru fiecare $y_i = H(x_i)$;
 - ▶ Caută $i \neq j$ cu $H(x_i) = H(x_j)$;
 - ▶ Dacă nu găsește nici o coliziune, reia atacul.
- ▶ Cum probabilitatea de succes a atacului este $\geq 1/2$, atunci numărul de încercări este ≈ 2 .

Transformarea Merkle-Damgård

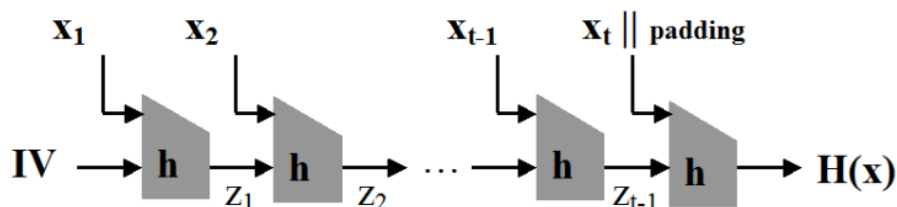
- ▶ Numim **funcție de compresie** o funcție hash rezistentă la coliziuni de intrare de lungime fixă;
- ▶ **Întrebare:** Intuitiv, ce se construiește mai ușor, H_1 sau H_2 ?

$$H_1 : \{0, 1\}^{m_1} \rightarrow \{0, 1\}^{n_1}, m_1 > n_1, m_1 \approx n_1$$

$$H_2 : \{0, 1\}^{m_2} \rightarrow \{0, 1\}^{n_2}, m_2 \gg n_2$$

- ▶ **Răspuns:** Cu cât domeniul și codomeniul funcției sunt mai apropiate ca dimensiune, numărul de coliziuni scade $\Rightarrow$ coliziunile devin mai dificil de determinat (considerăm că funcția distribuie valorile uniform aleator).
-

- Scopul este să construim o funcție hash (cu intrare de lungime variabilă), pornind de la o funcție de compresie (de lungime fixă);
- Pentru aceasta se aplică **transformarea Merkle-Damgård**;



Dacă h prezintă rezistență la coliziuni, atunci și H prezintă rezistență la coliziuni.

► MD5 (Message Digest 5)

- definit în 1991 de R. Rivest ca înlocuitor pentru MD4
- produce o secvență hash de 128 biți
- nesigur din 1996
- utilizat în versiuni mai vechi de Moodle

► SHA (Secure Hash Algorithm)

- o familie de funcții hash publicate de NIST
- SHA-0 și SHA-1 sunt nesigure
- SHA-2 prezintă 2 variante: SHA-256 și SHA-512
- SHA-3 adoptat în 2012 pe baza Keccak

- Este $H(x) = x \pmod{N}$ o funcție hash rezistentă la coliziuni? Dacă da, explicați de ce, dacă nu, găsiți o coliziune.

- **Răspuns:** Funcția nu e rezistentă la coliziuni: pentru orice $x < N$ și orice $a \in \mathbb{N}$, x și $aN + x$ formează o coliziune.

- Este $H(x) = ax + b \pmod{N}$ cu $(a, N) = 1$ o funcție hash rezistentă la coliziuni? Dacă da, explicați de ce, dacă nu, găsiți o coliziune.

- **Răspuns:** Funcția nu e rezistentă la coliziuni: cautăm x_1 și x_2 așa încât $H(x_1) = H(x_2)$ adică $ax_1 + b = ax_2 + b \pmod{N}$. Obținem $a(x_1 - x_2) = 0 \pmod{N}$. Cum $(a, N) = 1$ obținem că $x_1 - x_2 = 0 \pmod{N}$.

Curs 8 – SHA 3, Aplicații ale funcțiilor hash, Criptografie cu cheie publică (asimetrică), Teoria numerelor pentru criptografie, Prezumpții criptografice dificile

- **MD5** ► output pe 128 biți ► propusă în 1991 și considerată rezistentă la coliziuni o perioadă de timp
- 2004 - atac pentru găsirea de coliziuni + coliziuni explicite ► astăzi se pot găsi coliziuni în mai puțin de un minut pe un calculator desktop

► SHA-1

- face parte din seria de algoritmi standardizați SHA (Secure Hash Algorithm) și a fost standardul aprobat de NIST până în 2011
- output pe 160 biți
- 2005 - atac teoretic pentru găsirea coliziunilor; necesită 2^{69} evaluări ale funcției hash
- 2017 - prima coliziune practică - 2^{63} evaluări ale funcției hash
- atac pentru găsirea de *coliziuni cu prefix* - aprox. 2^{67} evaluări
- *coliziuni cu prefix* - pornind de la prefixele P și P' , se cere găsirea mesajelor $M \neq M'$ cu $H(P||M) = H(P'||M')$
- în practică, acestea sunt mai periculoase, pot duce la diverse atacuri incluzând generarea de certificate digitale false și atacuri asupra TLS, SSH

► **SHA-2** ► prezintă două versiuni: SHA-256 și SHA-512 în funcție de lungimea output-ului ► nu se cunosc vulnerabilități; atât SHA-2 cât și SHA-3 sunt sigure de folosit acolo unde rezistența la coliziuni este necesară

Criterii de selecție sha3 ► Lungimea secvenței de ieșire: $n = 224, 256, 384$ și 512 biți; ► Alte dimensiuni ale secvenței de ieșire sunt opționale; ► Eficiența crescută față de SHA-2; ► Utilizare în HMAC; ► Rezistența la coliziuni, prima și a doua preimagine (conform cu atacurile generice, tradiționale); ► Demonstratie de securitate; ► Parametrizabilă, număr de runde variabil; ► Simplitate, claritate. ► Keccak este câștigătorul competiției SHA-3 ► SHA-2 rămâne în continuare sigur

Atac pentru găsirea de coliziuni

- Considerăm funcții hash

$$H : \{0, 1\}^* \rightarrow \{0, 1\}^n$$

- **Rețineți!** Cel mai bun atac generic pentru găsirea de coliziuni ne spune că avem nevoie de funcții hash cu output-ul pe $2n$ biți pentru securitate împotriva adversarilor care rulează în timp 2^n ...
- ... spre deosebire de sistemele de criptare bloc, unde avem nevoie de n biți pentru securitate împotriva adversarilor care rulează în timp 2^n
- **Exemplu.** Dacă dorim ca găsirea de coliziuni să necesite timp 2^{128} atunci trebuie să alegem funcții hash cu output-ul pe 256 biți
- Am văzut că funcțiile hash pot fi folosite pentru a construi MAC-uri de lungime variabilă (în HMAC)

$$x \rightarrow H(x) \rightarrow \text{Mac}_k(H(x))$$

pentru MAC de lungime fixă

- Funcțiile hash au și multe alte aplicații atât în criptografie cât și în securitate
- Pentru un fișier x , $H(x)$ poate servi ca identificator unic (amprentă) - existența unui fișier diferit cu același hash implică găsirea de coliziuni în H .
- Prezintă câteva aplicații care decurg de aici

► Amprentarea virusilor

► scanner-ele de virusi pastreaza hash-urile virusilor cunoscuti

► verificarea fisierelor potential malitioase se face comparand hash-ul lor cu hash-ul virusilor cunoscuti.

► Deduplicarea datelor - stocarea in cloud partajata de mai multi utilizatori - se verifica daca hash-ul unui fisier nou (de incarcat) exista deja in cloud, caz in care se adauga doar un pointer la fisierul respective

Arbori Merkle

1. Prima solutie:

► clientul stochează local $h_1 = H(x_1), \dots, h_n = H(x_n)$ și verifică dacă $h_i = H(x'_i)$ pentru fiecare fișier x'_i recuperat

► **Dezavantaj:** spațiul necesar stocării crește liniar în n

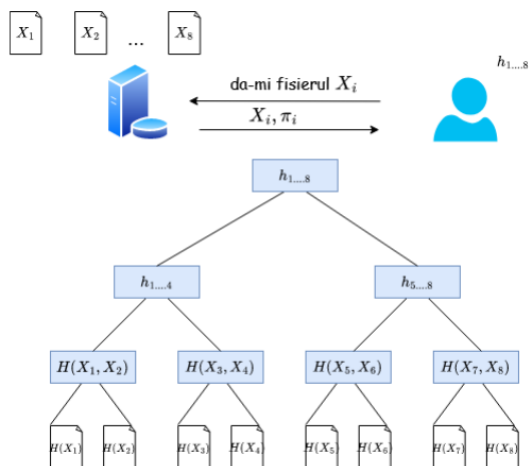
2. A doua soluție

► clientul stochează local un singur hash $h = H(x_1, \dots, x_n)$

► **Dezavantaj:** pentru a verifica x_i , clientul trebuie să recupereze toate fișierele $x_1, \dots, x_n$

► **Soluție:** arborii Merkle oferă un compromis între cele două soluții de mai sus

Arbori Merkle - stocarea fișierelor în cloud



► π_i - conține nodurile adiacente drumului de la X_i la rădăcină

► pe baza lui π_i , user-ul poate re-calcula valoarea rădăcinii pentru a verifica dacă este aceeași cu valoarea stocată de el

Stocarea parolelor ► Cea mai utilizata metoda de autentificare este bazata pe nume de utilizator si parola; ►

Metoda presupune o etapa de inregistrare, in care utilizatorul alege un username si o parola (pwd); ► Ulterior, la fiecare logare, utilizatorul introduce cele 2 valori; ► Daca username se regaseste in lista de utilizatori inregistrati si parola introdusa este corecta atunci autentificarea se realizeaza cu succes; ► In caz contrar, autentificarea esuaza.

► O greseala majora este stocarea parolelor in clar! ► Intrebare: De ce parolele NU trebuie stocate in clar? ►

Raspuns: Pentru ca daca adversarul capata acces la fisierul de parole atunci afla direct parolele tuturor utilizatorilor! ► Pentru stocarea parolelor se utilizeaza functiile hash; ► In fisierul de parole (sau baza de date) se stocheaza, pentru fiecare utilizator perechi de forma: (username, H(pwd))

► Pentru autentificare, utilizatorul introduce numele de utilizator username si parola pwd; ► Sistemul de autentificare calculeaza H(pwd) si se verifica daca valoarea obtinuta este stocata in fisierul de parole pentru utilizatorul indicat prin username; ► Daca da, atunci autentificarea se realizeaza cu succes; in caz contrar, autentificarea esuaza; ► Functiile hash sunt functii one-way: cunoscand H(pwd) nu se poate determina pwd; ► Aceasta metoda de stocare a parolelor introduce deci avantajul ca nu ofera adversarului acces direct la parole, chiar daca acesta detine fisierul de parole.

Atac de tip dictionar

► Adversarul poate insa sa verifice, pe rand, toate parolele cu probabilitate mare de aparitie; ► Acestea sunt de obicei cuvinte cu sens sau parole uzuale; ► Se considera ca formeaza termenii unui dictionar, de unde si numele atacului: atac de tip dictionar; ► Pentru a minimiza sansele unor astfel de atacuri: ► se blocheaza procesul de autentificare dupa un anumit numar de incercari nereusite; ► se obliga utilizatorul sa foloseasca o parola care satisface anumite criterii : o lungime minima, utilizarea a cel putin 3 tipuri de simboluri (litere mici, litere mari, cifre, caractere speciale); ► In caz de succes, adversarul determina parola unui singur utilizator;

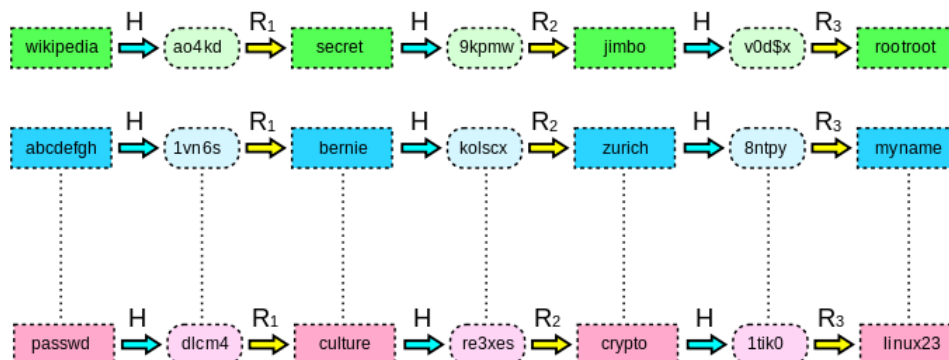
Atac folosind tabele hash precalculate

► Pentru a determina parolele mai multor utilizatori simultan, un adversar poate precalcuła valorile hash ale parolelor din dictionar; ► Dacă a adversarul capata acces la fisierul de parole, atunci verifica valorile care se regasesc in lista precalculata; ► Toate conturile utilizatorilor pentru care se potrivesc valorile sunt compromise; ► Atacul necesita capacitate de stocare mare: trebuie stocate toate perechile (pwd, H(pwd)) unde pwd este o parola din dictionar;

Rainbow tables

- **Rainbow tables** introduc un compromis între capacitatea de stocare și timp;
- Se compun lanțuri de lungime t :

$$pwd_1 \xrightarrow{H} H(pwd_1) \xrightarrow{f} pwd_2 \xrightarrow{H} H(pwd_2) \xrightarrow{f} \dots \xrightarrow{f} pwd_{t-1} \xrightarrow{H} H(pwd_{t-1}) \xrightarrow{f} pwd_t \xrightarrow{H} H(pwd_t)$$
- f este o funcție de mapare a valorilor hash în parole;
- Atenție! Funcția f nu este inversa funcției H (s-ar pierde proprietatea de *unidirecționalitate* a funcției hash)
- Se memorează doar capetele lanțurilor, valorile intermediare se generează la nevoie;
- Dacă t este suficient de mare, atunci capacitatea de stocare scade semnificativ:



- Algoritm de determinare a unei parole:
 1. Se caută valoarea hash a parolei în lista de valori hash a tabelii stocate;
 - 1.1 Dacă se găsește, atunci lanțul căutat este acesta și se trece la pasul 2;
 - 1.2 Dacă nu se găsește, se reduce valoarea hash prin funcția de reducere f într-o parolă căreia i se aplică funcția H și se reia cautarea de la pasul 1;
 2. Se generează întreg lanțul plecând de la valoarea inițială stocată. Parola corespunzătoare este cea situată în lanț înainte de valoarea hash căutată.

Salting

- ▶ Pentru a evita atacurile precedente, se utilizează tehnica de **salting**;
- ▶ La înregistrare, se stochează pentru fiecare utilizator:
 $(username, salt, H(pwd || salt))$
- ▶ *salt* este o secvență aleatoare de n biți, distinctă pentru fiecare utilizator;
- ▶ Adversarul nu poate precalcu valorile hash înainte de a obține acces la fișierul de parole...
- ▶ ... decât dacă folosește 2^n valori posibile *salt* pentru fiecare parolă;
- ▶ Atacurile devin deci impracticabile pentru n suficient de mare.

▶ În plus, în practică se folosesc funcții hash lente; ▶ Astfel verificarea unui număr mare de parole devine impracticabilă în timp real; ▶ Un alt avantaj introdus de tehnica de salting este că dacă 2 utilizatori folosesc aceeași parolă, valorile stocate sunt diferite: (Alice, 1652674, $H(\text{parolătest} || 1652674)$) (Bob, 3154083, $H(\text{parolătest} || 3154083)$) ▶ Prin simplă citire a fișierului de parole, adversarul nu își poate da seama că 2 utilizatori folosesc aceeași parolă.

Parole de unica folosință

▶ Cazul extrem de schimbare periodică a parolei îl reprezintă parolele de unica folosință; ▶ Utilizatorul folosește o listă de parole, la fiecare logare utilizând următoarea parolă din listă; ▶ Această listă se calculează pornind de la o valoare x folosind o funcție hash H ; ▶ Să considerăm o listă de 3 parole de unica folosință: $P_0 = H(H(H(x)))$, $P_1 = H(H(H(x)))$, $P_2 = H(H(x))$, $P_3 = H(x)$; ▶ La înregistrare, în fișierul de parole se păstrează tripletul: (username, 1, $P_0 = H(H(H(x)))$) ▶ Utilizatorul introduce o parolă P_1 și autentificarea se realizează cu succes dacă $H(P_1) = P_0$; ▶ Se actualizează fișierul de parole cu noua parolă introdusă: (username, 2, $P_1 = H(H(H(x)))$) ▶ Procesul continuă până se ajunge la $H(x)$; ▶ Fiind cunoscută o parolă din secvență, se poate calcula imediat parolă anterioară, dar NU se poate calcula parolă următoare.

Criptografie cu cheie publică (asimetrică)

Limitările criptografiei simetrice ▶ Am studiat până acum criptografia simetrică; ▶ Aceasta asigură confidențialitatea și integritatea mesajelor transmise pe canale nesecurizate; ▶ Însă rămân multe probleme nerezolvate...

Problema 1 - Distribuirea cheilor

▶ Criptarea simetrică necesită o cheie secretă comună partilor comunicante; ▶ Întrebare: Cum se obține și se distribuie aceste chei?

▶ Varianta 1. Se transmite printr-un canal de comunicație nesecurizat; ▶ NU! Un adversar pasiv le poate intercepta și utiliza ulterior pentru decriptarea comunicației.

▶ Varianta 2. Se transmite printr-un canal de comunicație sigur care presupune un serviciu de mesagerie de încredere; ▶ Opțiune posibilă la nivel guvernamental sau militar; ▶ Dar nu va fi niciodată posibilă în cazul organizațiilor numeroase; ▶ Presupunem doar cazul în care fiecare manager trebuie să partajeze o cheie secretă cu fiecare subordonat; ▶ Problemele care apar sunt multiple: pentru fiecare nou angajat este necesară stabilirea cheilor, organizația are sedii în mai multe țări, ...

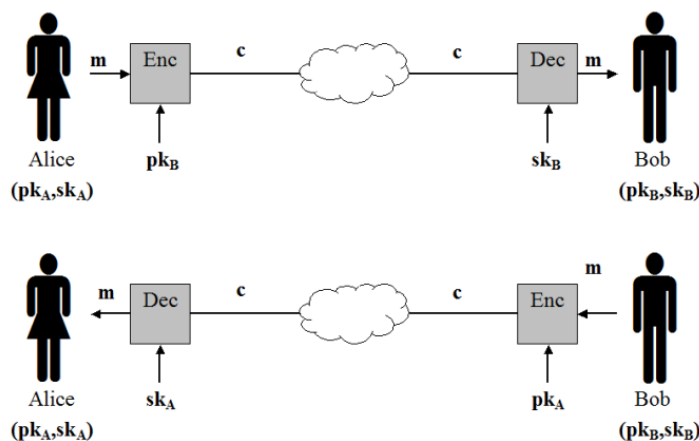
Problema 2 - Stocarea secretă a cheilor

- ▶ Rămânem la exemplul anterior al unei organizații numeroase cu N angajați;
- ▶ **Întrebare:** Câte chei sunt necesare pentru ca fiecare 2 angajați să poată comunica criptat?
- ▶ **Răspuns:** $C_N^2 = N(N - 1)/2$;
- ▶ La acestea se adaugă cheile necesare pentru accesul la resurse (servere, imprimante, baze de date ...);
- ▶ Apare o problemă de **logistică**: foarte multe chei sunt dificil de menținut și utilizat;
- ▶ Și apare o problemă de **securitate**: cu cât sunt mai multe chei, cu atât sunt mai dificil de stocat în mod sigur, deci cresc șansele de a fi furate de adversari;

Problema 3 - Medii de comunicare deschise ▶ Deși dificil de stocat sau utilizat, criptografia simetrică ar putea (cel puțin în teorie) să rezolve aceste probleme; ▶ Dar este insuficientă în medii deschise, în care participanții nu se întâlnesc niciodată; ▶ Astfel de exemple includ: o tranzacție prin internet sau un e-mail transmis unei persoane necunoscute;

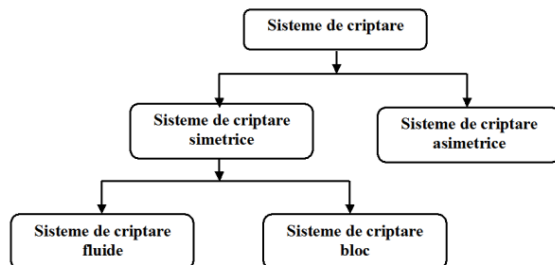
Problema 4 - Imposibilitatea non-repudierii ▶ O cheie simetrică este deținută de cel puțin 2 părți; ▶ Este imposibil de demonstrat de exemplu că un MAC a fost produs de una dintre cele 2 părți comunicante; ▶ De aceea nu se poate utiliza autentificarea simetrică pentru a atesta sursa unui mesaj sau document.

Criptografia asimetrică ▶ Criptografia cu cheie publică este introdusă de W.Diffie și M.Hellman în 1976 ca o soluție la problemele enumerate anterior:



Sisteme de criptare asimetrice

- ▶ Am studiat sisteme de criptare care folosesc **aceeași cheie** pentru criptare și decriptare - **sisteme de criptare simetrice**;
- ▶ Vom studia sisteme de criptare care folosesc **chei diferite** pentru criptare și decriptare - **sisteme de criptare asimetrice**;



Terminologie ▶ Spre deosebire de criptarea simetrică, criptarea asimetrică folosește o pereche de chei:
▶ Cheia publică p_k este folosită pentru criptare; ▶ Cheia secretă s_k este folosită pentru decriptare; ▶ Cheia publică este larg răspândită pentru a asigura posibilitatea de criptare oricui dorește să transmită un mesaj către entitatea careia îi corespunde; ▶ Cheia secretă este privată și nu se cunoaște decât de entitatea careia îi corespunde; ▶ Considerăm (pentru simplitate) ca ambele chei au lungime cel puțin n biți.

Criptografia simetrică

- ▶ necesită secretizarea întregii chei
- ▶ folosește aceeași cheie pentru criptare și decriptare
- ▶ rolurile emițătorului și ale receptorului pot fi schimbate
- ▶ pentru ca un utilizator să primească mesaje criptate de la mai mulți emițători, trebuie să partajeze cu fiecare câte o cheie

Avantaje

- ▶ număr mai mic de chei
- ▶ simplifică problema distribuirii cheilor
- ▶ fiecare participant trebuie să stocheze o singură cheie secretă de lungă durată
- ▶ permite comunicarea sigură pe canale publice
- ▶ rezolvă problema mediilor de comunicare deschise

Criptografia asimetrică

- ▶ necesită secretizarea unei jumătăți din cheie
- ▶ folosește chei distincte pentru criptare și decriptare
- ▶ rolurile emițătorului și ale receptorului nu pot fi schimbate
- ▶ o pereche de chei asimetrice permite oricui să transmită informație criptată către entitatea careia îi corespunde

Dezavantaje

- ▶ criptarea asimetrică este mult mai lentă decât criptarea simetrică
- ▶ compromiterea cheii private conduce la compromiterea tuturor mesajelor criptate primite, indiferent de sursă
- ▶ necesită verificarea autenticității cheii publice (PKI rezolvă această problemă)

Teoria numerelor pentru criptografie

- ▶ $\mathbb{Z} = \{\dots, -2, -1, 0, 1, 2, \dots\}$
- ▶ $\mathbb{N} = \{0, 1, 2, \dots\}$
- ▶ $\mathbb{Z}_+ = \{\dots, -2, -1, 0, 1, 2, \dots\}$
- ▶ Pentru $a, n \in \mathbb{N}$ notăm $\gcd(a, n)$ ca fiind cel mai mare divizor comun (greatest common divisor) al lui a și n .
- ▶ **Exemplu:** $\gcd(30, 50) = 10$.
- ▶ pentru $n \in \mathbb{Z}_+$ notăm
 - ▶ $\mathbb{Z}_n = \{0, 1, \dots, n-1\}$
 - ▶ $\mathbb{Z}_n^* = \{a \in \mathbb{Z}_n \mid \gcd(a, n) = 1\}$
 - ▶ $\phi(n) = |\mathbb{Z}_n^*|$
- ▶ **Exemplu:** $n=12$
 - ▶ $\mathbb{Z}_{12} = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11\}$
 - ▶ $\mathbb{Z}_{12}^* = \{1, 5, 7, 11\}$
 - ▶ $\phi(12) = 4$
- ▶ $a = qn + r$ cu $0 \leq r < n$.
Considerăm împărțirea lui a la n .
Atunci q este catul împărțirii iar r este restul și notăm

$$a \bmod n = r$$

- ▶ **Exemplu:** $17 \bmod 3 = 2$.
- ▶ $a \equiv b \pmod n$ dacă $a \bmod n = b \bmod n$.

Grupuri și invers

- ▶ Dacă $n \in \mathbb{Z}_+$ atunci $G = \mathbb{Z}_n^*$ împreună cu operația "*" definită

$$a * b = ab \bmod n$$

pentru $a, b \in G$ formează un grup și are cele trei proprietăți:

- ▶ asociativitatea: operația $*$ este asociativă
- ▶ element neutru: există un element $1 \in G$ așa încât $a * 1 \bmod n = 1 * a \bmod n = a, \forall a \in G$.
- ▶ element inversabil: pentru orice $a \in G$ există un unic $b \in G$ așa încât $a * b = b * a = 1 \bmod n$.
 b se numește inversul lui a și îl notăm cu $a^{-1} \bmod n$.
- ▶ **Exemplu:** $5^{-1} \bmod 12$ este acel număr $b \in G$ care satisface $5b \bmod 12 = 1$
- ▶ deci $b = 5$.

► calculati $5 * 8 * 10 * 16 \bmod 21$. ► Prima varianta: Calculam mai intai $5 * 8 * 10 * 16 = 6400$ si apoi calculam $6400 \bmod 21 = 16$ ► A doua varianta (mai rapida): ► $5 * 8 \bmod 21 = 40 \bmod 21 = 19$ ► $19 * 10 \bmod 21 = 190 \bmod 21 = 1$ ► $1 * 16 \bmod 21 = 16$

Ordinul unui grup

- Ordinul unui grup G este numărul de elemente din acel grup, îl notăm cu $|G|$.

- **Exemplu:** Ordinul lui $\mathbb{Z}_{21}^*$ = 12 pentru că

$$\mathbb{Z}_{21}^* = \{1, 2, 4, 5, 8, 10, 11, 13, 16, 17, 19, 20\}$$

- Fie G un grup de ordin m si $a \in G$. Atunci:

- $a^m = 1$.

- Pentru orice $i \in \mathbb{Z}$, $a^i = a^{i \bmod m}$.

- **Exemplu:** Calculați $5^{74} \bmod 21$.

- **Răspuns:** Fie $\mathbb{Z}_{21}^*$ si $a = 5$. Atunci $m = 12$ și

- $5^{74} \bmod 21 = 5^{74 \bmod 12} \bmod 21 = 5^2 \bmod 21 = 4$.

Prezumtii criptografice dificile

Prezumții criptografice dificile

- Criptografia modernă se bazează pe presupunția că anumite probleme nu pot fi rezolvate în timp polinomial;
- Până acum am văzut că schemele de criptare și de autentificare se bazează pe presupunția existenței permutărilor pseudoaleatoare;
- Dar această presupunție e nenaturală și foarte puternică;
- În practică, PRF pot fi instanțiate cu cifruri bloc;
- Însă metode pentru a demonstra pseudoaleatorismul construcțiilor practice relativ la alte presupunții "mai rezonabile" nu se cunosc;
- Dar e posibil a demonstra existența permutărilor pseudoaleatoare pe baza unei presupunții mult mai slabe, cea a existenței funcțiilor one-way;
- In continuare vom introduce cateva doua considerate "dificile": **problema factorizarii** si **problema logaritmului discret** si vom prezenta functii conjecturate ca fiind one-way bazate pe aceste probleme; ► Tot materialul ce urmeaza se bazeaza pe notiuni de teoria numerelor; ► La criptografia simetrica (cu cheie secreta) am vazut primitive criptografice (i.e. functii hash, PRG, PRF) care pot fi construite eficient fara a implica teoria numerelor; ► La criptografia asimetrica (cu cheie publica) constructiile cunoscute se bazeaza pe probleme matematice dificile din teoria numerelor;

- ▶ O primă problemă conjecturată ca fiind dificilă este **problema factorizării numerelor întregi** sau mai simplu **problema factorizării**;
- ▶ Fiind dat un număr compus N , problema cere să se găsească două numere prime x_1 și x_2 pe n biți așa încât $N = x_1 \cdot x_2$;
- ▶ Cele mai greu de factorizat sunt numerele care au factori primi foarte mari.
- ▶ Pentru a putea folosi problema în criptografie, trebuie să generăm numere prime aleatoare în mod eficient; ▶ Putem genera un număr prim aleator pe n biți prin alegerea repetată de numere aleatoare pe n biți până când găsim unul prim; ▶ Pentru eficiență, ne interesează două aspecte: 1. probabilitatea ca un număr aleator de n biți să fie prim; 2. cum testăm eficient ca un număr dat p este prim.
- ▶ Pentru distribuția numerelor prime, se cunoaște următorul rezultat matematic:

Teoremă

Pentru orice $n > 1$, proporția de numere prime pe n biți este de cel puțin $1/3n$.

- ▶ Rezultă imediat că dacă testăm $t = 3n^2$ numere, probabilitatea ca un număr prim să nu fie ales este cel mult e^{-n} , deci neglijabilă.
 - ▶ Deci avem un algoritm în timp polinomial pentru găsirea numerelor prime
- Testarea primalității** ▶ Cei mai eficienți algoritmi sunt probabilisti: ▶ Dacă numărul p dat este prim atunci algoritmul întotdeauna întoarce rezultatul prim; ▶ Dacă p este compus, atunci cu probabilitate mare algoritmul va întoarce compus; ▶ Concluzie: dacă outputul este compus, atunci p sigur este compus, dacă outputul este prim, atunci cu probabilitate mare p este prim dar este posibil să se fi produs o eroare; ▶ Un algoritm determinist polinomial a fost propus în 2002, dar este mai lent decât algoritmi probabilisti; ▶ Un algoritm probabilist foarte răspândit este Miller-Rabin care accepta la intrare un număr N și rulează în timp polinomial în $|N|$
- ▶ **Deocamdată nu se cunosc algoritmi polinomiali** pentru problema factorizării; ▶ Dar există algoritmi mult mai buni decât forța brută; ▶ Prezentăm în continuare câțiva algoritmi de factorizare.

- ▶ **Reamintim:** Fiind dat un număr compus N , **problema factorizării** cere să se găsească 2 numere prime p și q a.î. $N = pq$;
- ▶ Considerăm $|p| = |q| = n$ și deci $n = O(\log N)$;
- ▶ Metoda cea mai simplă este împărțirea numărului N prin toate numerele p impare din intervalul $p = 3, \dots, \lfloor \sqrt{N} \rfloor$.
- ▶ Complexitatea timp este $O(\sqrt{N} \cdot (\log N)^c)$ unde c este o constantă, adică exponențială în numărul b de biți al lui N ; $b = \log_2 N$. Complexitatea este $O(N^{1/2}) = O((2^{\log_2 N})^{1/2})$
- ▶ Pentru $N < 10^{12}$ metoda este destul de eficientă.

- ▶ Există însă algoritmi mai sofisticăți, cu timp de execuție mai bun, între care:
 - ▶ Metoda **Pollard p – 1**: funcționează atunci când $p - 1$ are factori primi "mici";
 - ▶ Metoda **Pollard rho**: timpul de execuție este $O(N^{1/4} \cdot (\log N)^c)$, deci tot exponențial în lungimea lui N ;
 - ▶ **Algoritmul sitei pătratice** - rulează în timp *sub-exponențial* în lungimea lui N .
- ▶ Deocamdată, cel mai rapid algoritm de factorizare este o îmbunătățire a sitei pătratice care factorizează un număr N de lungime $O(n)$ în timp $2^{O(n^{1/3} \cdot (\log n)^{2/3})}$.

RSA Challenge ▶ În 1991, Laboratoarele RSA lansează RSA Challenge; ▶ Aceasta presupune factorizarea unor numere N , unde $N = p \cdot q$, cu p, q 2 numere prime mari; ▶ Au fost lansate mai multe provocări, câte 1 pentru fiecare dimensiune (în biti) a lui N : ▶ Exemple includ: RSA-576, RSA-640, RSA-768, ..., RSA-1024, RSA-1536, RSA-2048; ▶ Provocarea s-a încheiat oficial în 2007; ▶ Multe provocări au fost sparte în cursul anilor (chiar și ulterior închiderii oficiale), însă există numere încă nefactorizate:

2. Problema logaritmului discret

- ▶ O altă prezumție dificilă este **DLP (Discrete Logarithm Problem)** (sau **PLD (Problema Logaritmului Discret)**);
- ▶ Considerăm $\mathbb{G}$ un grup ciclic de ordin q ;
- ▶ Există un generator $g \in \mathbb{G}$ a.î. $\mathbb{G} = \{g^0, g^1, \dots, g^{q-1}\}$;
- ▶ Echivalent, pentru fiecare $h \in \mathbb{G}$ există un *unic* $x \in \mathbb{Z}_q$ a.î. $g^x = h$;
- ▶ x se numește **logaritmul discret** al lui h în raport cu g și se notează

$$x = \log_g h$$

Experimentul logaritmului discret $DLog_{\mathcal{A}}(n)$

1. Generează $(\mathbb{G}, q, g)$ unde $\mathbb{G}$ este un grup ciclic de ordin q (cu $|q| = n$) iar g este un generator al lui $\mathbb{G}$.
2. Alege $h \xleftarrow{R} \mathbb{G}$. (se poate alege $x' \xleftarrow{R} \mathbb{Z}_q$ și apoi $h := g^{x'}$.)
3. $\mathcal{A}$ primește $\mathbb{G}, q, g, h$ și întoarce $x \in \mathbb{Z}_q$;
4. Output-ul experimentului este 1 dacă $g^x = h$ și 0 altfel.

Definiție

Spunem că problema logaritmului discret (DLP) este dificilă dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât

$$\Pr[DLog_{\mathcal{A}}(n) = 1] \leq \text{negl}(n)$$

Grupuri ciclice de ordin prim ► Exista cateva clase de grupuri ciclice pentru care DLP este considerata dificila; ► Una dintre ele este clasa grupurilor ciclice de ordin prim (in aceste grupuri, problema este "cea mai dificila"); ► DLP nu poate fi rezolvata in timp polinomial in grupurile care nu sunt de ordin prim, ci doar este mai usoara; ► In aceste grupuri cautarea unui generator si verificarea ca un numar dat este generator sunt triviale.

- DLP este considerată dificilă în grupuri ciclice de forma $\mathbb{Z}_p^*$ cu p prim;
- Inșă pentru $p > 3$ grupul $\mathbb{Z}_p^*$ NU are ordin prim;
- Aceasta problemă se rezolvă folosind un *subgrup* potrivit al lui $\mathbb{Z}_p^*$;

► Cel mai rapid algoritm de factorizare necesita timp sub-exponential; ► Problema logaritmului discret este dificila.

Curs 9 - Notiuni de securitate in criptografia asimetrica, Criptare hibrida, RSA, PKCS

Securitate perfecta ► Incepem studiul securitatii in acelasi mod in care am inceput la criptografia simetrica: cu securitatea perfecta; ► Definitia e analoaga cu diferenta ca adversarul cunoaste, in afara textului criptat, si cheia publica;

- **Întrebare:** Securitatea perfectă este posibilă în cadrul criptografiei cu cheie publică?
- **Răspuns:** NU! Indiferent lungimea cheilor și a mesajelor;
- Având pk și un text criptat $c = Enc_{pk}(m)$, un adversar nelimitat computațional poate determina mesajul m cu probabilitate 1.

Indistinctibilitatea

- Indistinctibilitatea în criptografia cu cheie publică este corespondenta noțiunii similare din criptografia cu cheie secretă;
- Vom defini această noțiune pe baza unui experiment de indistinctibilitate $PubK_{\mathcal{A},\pi}^{eav}(n)$ unde $\pi = (Enc, Dec)$ este schema de criptare iar n este parametrul de securitate al schemei π ;
- Personaje participante: **adversarul** $\mathcal{A}$ care încearcă să spargă schema și un **challenger**.

Definiție

O schemă de criptare $\pi = (Enc, Dec)$ este indistinctibilă în prezența unui atacator pasiv dacă pentru orice adversar $\mathcal{A}$ există o funcție neglijabilă $negl$ așa încât

$$Pr[PubK_{\mathcal{A},\pi}^{eav}(n) = 1] \leq \frac{1}{2} + negl(n).$$

Securitate pentru interceptare simplă

- ▶ Principala diferență față de definiția similară studiată la criptografia cu cheie secretă este că $\mathcal{A}$ primește cheia publică pk ;
- ▶ Adică $\mathcal{A}$ primește acces *gratuit* la un oracol de criptare, ceea ce înseamnă că el poate calcula $Enc_{pk}(m)$ pentru orice m ;
- ▶ Prin urmare, definiția este echivalentă cu cea pentru securitate CPA (nu mai este necesar oracolul de criptare pentru că $\mathcal{A}$ își poate cripta singur mesajele);
- ▶ Reamintim că în criptografia simetrică există scheme indistinctibil sigure dar care nu sunt CPA-sigure .

Insecuritatea schemelor deterministe

- ▶ Dupa cum am vazut la criptografia simetrica, nici o schema determinista nu poate fi CPA sigura;
- ▶ Datorita echivalentei intre notiunile de securitate CPA si indistinctibilitate pentru interceptare simpla (in criptografia asimetrica) concluzionam ca: **Teorema** Nici o schema de criptare cu cheie publica determinista nu poate fi semantic sigura pentru interceptarea simpla.

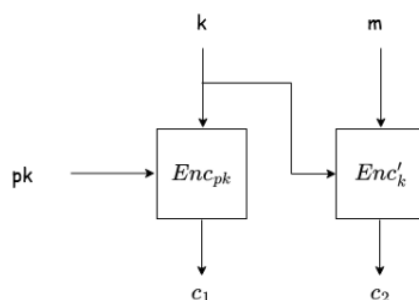
Securitate CCA ▶ Notiunea de securitate CCA ramane identica cu cea de la sistemele simetrice; ▶ Capabilitatile adversarului: el poate interactiona cu un oracol de decriptare, fiind un adversar activ care poate rula atacuri in timp polinomial; ▶ Adversarul poate transmite catre oracolul de decriptare anumite mesaje c si primeste inapoi mesajul clar corespunzator; ▶ Ca si in cazul securitatii CPA, adversarul nu mai necesita acces la oracolul de criptare pentru ca detine cheia publica pk si poate realiza singur criptarea oricarui mesaj m .

▶ **In criptografia cu cheie publica: ▶ NU exista securitate perfecta ▶ indistinctibilitate = securitate CPA**

Criptarea hibrida

- ▶ Criptarea cu cheie secreta este mult mai rapida decat criptarea cu cheie publica
- ▶ Pentru mesajele care sunt suficient de lungi, se foloseste criptare cu cheie secreta in tandem cu criptarea cu cheie publica;

- ▶ Rezultatul acestei combinații se numește **criptare hibridă** și este folosită extensiv în practică;



- ▶ Pentru criptarea unui mesaj m , se urmează doi pași:
- 1. Expeditorul alege aleator o cheie k pe care o criptează folosind cheia publică a destinatarului, rezultând $c_1 = Enc_{pk}(k)$; Numai destinatarul va putea decripta k , ea rămânând secretă pentru un adversar;
- 2. Expeditorul criptează m folosind o schemă de criptare cu cheie secretă (Enc', Dec') cu cheia k , rezultând $c_2 = Enc'_k(m)$;
- ▶ Mesajul criptat este $c = (c_1, c_2)$;

Teoremă

Dacă Π este o schemă de criptare cu cheie publică CPA-sigură iar Π' este o schemă de criptare cu cheie secretă sigură semantic, atunci construcția hibridă Π^{hyb} este o schemă de criptare cu cheie publică CPA-sigură.

- ▶ Este suficient ca Π' să satisfacă noțiunea mai slabă de securitate semantică (care nu implică securitate CPA)...
 - ▶ ...deoarece cheia secretă k este una "nouă" și aleasă aleator de fiecare dată când se criptează un mesaj;
 - ▶ Cum o cheie k este folosită o singură dată, e suficientă noțiunea de securitate la interceptare simplă pentru securitatea schemei hibride.
- ▶ Pentru criptarea mesajelor lungi, în practică se folosește criptarea hibridă ▶ Aceasta îmbină avantajele criptării simetrice și criptării asimetrice

RSA

Funcții one-way

- ▶ Reprezintă o primitivă criptografică minimă, necesară și suficientă pentru criptarea cu cheie secretă dar și pentru codurile de autentificare a mesajelor
- ▶ O funcție f one-way este ușor de calculat și dificil de inversat

Definiție

O funcție $f : \{0, 1\}^* \rightarrow \{0, 1\}^*$ este one-way dacă următoarele două condiții sunt îndeplinite:

1. Ușor de calculat: Există un algoritm polinomial pentru calculul lui f
2. Dificil de inversat: Pentru orice algoritm polinomial A , există o funcție neglijabilă $negl$ așa încât

$$\Pr[\text{Invert}_{A,f}(n) = 1] \leq \text{negl}(n)$$

- ▶ Problema factorizării numerelor mari în produs de două numere prime de aceeași lungime este one-way ... ▶ ... însă nu poate fi folosită direct pentru criptografie ▶ În schimb, introducem o problemă apropiată de problema factorizării pe baza căreia putem construi sisteme de criptare

Problema RSA

- ▶ Problema RSA se bazează pe dificultatea factorizării numerelor mari: $N = p \cdot q$, p și q prime;
- ▶ Fie $\mathbb{Z}_N^*$ un grup de ordin $\phi(N) = (p-1)(q-1)$;
- ▶ Dacă se cunoaște factorizarea lui N , atunci $\phi(N)$ este ușor de calculat
- ▶ Fixăm e cu $\gcd(e, \phi(N)) = 1$. Atunci $(x^e)^d = x^{ed \bmod \phi(N)} = x \bmod N = \bmod N = (x^d)^e$
- ▶ x^d se numește rădăcina de ordin e a lui x modulo N
- ▶ Dacă p și q se cunosc, atunci putem calcula $\phi(N)$ și $d = e^{-1} \bmod \phi(N)$
- ▶ Dacă p și q nu se cunosc
 - ▶ calculul lui $\phi(N)$ este la fel de dificil precum factorizarea lui N
 - ▶ calculul lui d este la fel de dificil precum factorizarea lui N

Experimentul RSA $RSA - inv_{\mathcal{A}, GenRSA(n)}$

- ▶ Considerăm algoritmul $GenRSA(1^n)$ care are ca output (N, e, d) unde $ed = 1 \bmod \phi(N)$
- ▶ Considerăm experimentul RSA pentru un algoritm $\mathcal{A}$ și un parametru n .
 1. Execută $GenRSA$ și obține (N, e, d) ;
 2. Alege $y \leftarrow \mathbb{Z}_N^*$;
 3. $\mathcal{A}$ primește N, e, y și întoarce $x \in \mathbb{Z}_N^*$;
 4. Output-ul experimentului este 1 dacă $x^e = y \bmod N$ și 0 altfel.

Definiție

Spunem că problema RSA este dificilă cu privire la $GenRSA$ dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă $negl$ așa încât

$$Pr[RSA - inv_{\mathcal{A}, GenRSA(n)} = 1] \leq negl(n)$$

- ▶ Prezumția RSA este că există un algoritm $GenRSA$ pentru care problema RSA este dificilă;
- ▶ Un algoritm $GenRSA$ poate fi construit pe baza unui număr compus împreună cu factorizarea lui;

Algorithm 1 $GenRSA$

Input: n

Output: N, e, d

- 1: **genereaza** p și q prime pe n -biti; $N = p \cdot q$
 - 2: $\phi(N) = (p - 1)(q - 1)$
 - 3: **gasește** e a.î. $\gcd(e, \phi(N)) = 1$
 - 4: **calculează** $d := e^{-1} \bmod \phi(N)$
 - 5: **return** N, e, d
-

- ▶ Valoarea lui e aleasa pare ca nu afecteaza dificultatea problemei RSA
- ▶ în practică se folosește $e = 3$ sau $e = 16$ pentru exponențiere eficientă
- ▶ Dacă N este ușor de factorizat, atunci problema RSA este ușoară
- ▶ Pentru ca problema RSA să poată fi dificilă, trebuie ca N -ul ales în $GenRSA$ să fie dificil de factorizat în produs de două numere prime;
- ▶ Nu se cunoaște nici o dovadă că nu există o altă metodă de a rezolva problema RSA care să nu implice calculul lui $\phi(N)$ sau al lui d .

Exemplu

- ▶ Presupunem $(N, p, q) = (143, 11, 13)$. Atunci $\phi(N) = 120$.
- ▶ Alegem e asa incat $\gcd(e, \phi(N)) = 1$, fie $e = 7$.
- ▶ Calculăm $d = e^{-1} \bmod \phi(N)$ si obtinem $d = 103$. Deci output-ul algoritmului GenRSA este $(143, 7, 103)$.

Textbook RSA

- ▶ Definim sistemul de criptare *Textbook RSA* pe baza problemei prezentată anterior;
 1. Se rulează GenRSA pentru a determina N, e, d .
 - ▶ Cheia publică este: $pk = (N, e)$;
 - ▶ Cheia privată este $sk = d$;
 2. **Enc:** dată o cheie publică (N, e) și un mesaj $m \in \mathbb{Z}_N$, întoarce $c = m^e \bmod N$;
 3. **Dec:** dată o cheie secretă (N, d) și un mesaj criptat $c \in \mathbb{Z}_N$, întoarce $m = c^d \bmod N$.
- ▶ Sistemul de criptare este corect pentru ca
 $\text{Dec}_{sk}(\text{Enc}_{pk}(m)) = m$ astfel:
 $(m^e)^d \bmod N = m^{ed \bmod \phi(N)} \bmod N = m^1 \bmod N = m$

Problema 1: Determinismul

▶ Intrebare: Este Textbook RSA CPA-sigur sau CCA-sigur? ▶ Raspuns: NU! Sistemul este determinist, deci nu poate rezista definitiilor de securitate!

Problema 2: Utilizarea multiplă a modulului

- ▶ Cunoscând e, d, N cu $(e, \phi(N)) = 1$ se poate determina eficient factorizarea lui N ;
- ▶ **Întrebare:** Este corect să se utilizeze mai multe perechi de chei care folosesc același modul?
- ▶ **Răspuns:** NU! Fie 2 perechi de chei:

$$pk_1 = (N, e_1); sk_1 = (N, d_1)$$

$$pk_2 = (N, e_2); sk_2 = (N, d_2)$$
- ▶ Posesorul perechii (pk_1, sk_1) factorizează N , apoi determină $d_2 = e_2^{-1} \bmod \phi(N)$.

▶ RSA este cel mai cunoscut si mai utilizat algoritm cu cheie publica; ▶ Textbook RSA NU trebuie utilizat!

PKCS

Padded RSA!

▶ Am vazut ca Textbook RSA este nesigur; ▶ Eliminam una dintre problemele principale, aceea este ca sistemul este determinist; ▶ Introducem in acest sens Padded RSA; ▶ Ideea este sa se adauge un numar aleator (pad) la mesajul clar inainte de criptare; ▶ Notam in continuare cu n parametrul de securitate (conform GenRSA).

1. Se rulează GenRSA pentru a determina N, e, d .
 - ▶ Cheia publică este: (N, e) ;
 - ▶ Cheia privată este (N, d) ;
2. **Enc:** dată o cheie publică (N, e) și un mesaj $m \in \{0, 1\}^{l(n)}$, alege $r \leftarrow^R \{0, 1\}^{N-l(n)-1}$, interpretează $r||m$ ca un element în $\mathbb{Z}_N$ și întoarce $c = (r||m)^e \bmod N$;
3. **Dec:** dată o cheie secretă (N, d) și un mesaj criptat $c \in \mathbb{Z}_N$, calculează $c^d \bmod N$ și întoarce ultimii $l(n)$ biți.
 - ▶ Pentru $l(n)$ foarte mare, atunci este posibil un atac prin forță brută care verifică toate valorile posibile pentru r ;
 - ▶ Pentru $l(n)$ mic se obține securitate CPA:

Teoremă

Dacă problema RSA este dificilă, atunci Padded RSA cu $l(n) = O(\log n)$ este CPA-sigură.

- ▶ martie 1998 - Laboratoarele RSA introduc PKCS #1 v1.5; ▶ PKCS = Public-Key Cryptography Standard;
- ▶ Folosește Padded RSA; ▶ Utilizat în HTTPS, SSL/TLS, XML Encryption.

PKCS #1 v1.5

- ▶ Notăm k lungimea modulului N în bytes: $2^{8(k-1)} \leq N < 2^{8k}$;
- ▶ Mesajele m care se criptează se consideră multipli de 8 biți, de maxim $k - 11$ bytes;
- ▶ Criptarea se realizează astfel:

$$(00000000||00000010||r||00000000||m)^e \bmod N$$

- ▶ r este ales aleator, pe $k - D - 3$ bytes nenuli, unde D este lungimea lui m în bytes;

Securitatea PKCS #1 v1.5

- ▶ Se crede că este CPA-sigur, dar acest lucru nu este demonstrat;
- ▶ Cu siguranță însă nu este CCA-sigur;
- ▶ În 1998, D.Bleichenbacher publică un atac care bazându-se pe faptul că serverul web (HTTPS) întoarce eroare dacă primii 2 octeți nu sunt **02**;
- ▶ Scopul adversarului este să decripteze un text c ;
- ▶ Adversarul transmite către server $c' = r^e \cdot c \bmod N$;
- ▶ Răspunsul serverului indică adversarului dacă c' este valid (i.e. începe cu **02**);
- ▶ Adversarul folosește răspunsul primit pentru a determina informații despre m ;
- ▶ Repetă atacul până determină mesajul m .

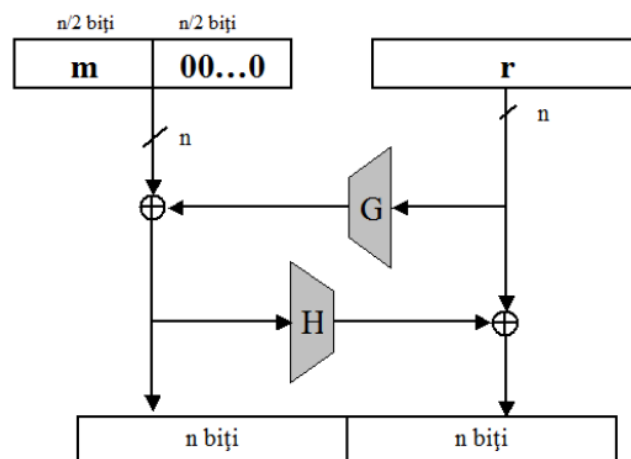
OAEP

- ▶ octombrie 1998 - Laboratoarele RSA introduc un nou standard PKCS demonstrat CCA-sigur...
- ▶ ...în modelul $\mathcal{ROM}$ (Random Oracle Model);
- ▶ Este vorba despre PKCS #1 v2.0 sau **OAEP** = **O**ptimal **A**symmetric **E**ncryption **S**tandard;
- ▶ OAEP este de fapt o modalitate de padding;
- ▶ OAEP este o metodă **nedeterministă** și **inversabilă** care transformă un mesaj m de lungime $n/2$ într-o secvență m' de lungime $2n$;
- ▶ Notăm $m' = OAEP(m, r)$, unde r este o secvență aleatoare (de lungime n);
- ▶ RSA-OAEP criptează $m \in \{0, 1\}^{n/2}$ folosind cheia publică (N, e) ca:

$$(OAEP(m, r))^e \mod N$$

- ▶ RSA-OAEP decriptează c folosind cheia secretă (N, d) ca:

$$(m, r) = OAEP^{-1}(c^d \mod N)$$



- ▶ G, H = funcții hash
- ▶ $m \in \{0, 1\}^{n/2}$ mesajul
- ▶ $r \leftarrow^R \{0, 1\}^n$
- ▶ se obține $OAEP(m, r)$ pe $2n$ biți

► Utilizarea RSA în practică: PKCS #1 v1.5 și PKCS #1 v2.0 (OAEP)

Curs 10 - Problema logaritmului discret, Schimbul de chei Diffie-Hellman, ElGamal, Criptografia bazată pe curbe eliptice, Schimbul de chei Diffie-Hellman și ElGamal pe curbe eliptice

- ▶ Reamintim PLD:
- ▶ Fie $\mathbb{G}$ un grup ciclic de ordin q (cu $|\mathbb{G}| = q$) iar g este generatorul lui $\mathbb{G}$.
- ▶ Pentru fiecare $h \in \mathbb{G}$ există un unic $x \in \mathbb{Z}_q$ a.î. $g^x = h$.
- ▶ PLD cere găsirea lui x știind $\mathbb{G}, q, g, h$; notăm $x = \log_g h$;
- ▶ Atenție! Atunci când $g^{x'} = h$ pentru un x' arbitrar (deci NU neapărat $x \in \mathbb{Z}_q$), notăm $\log_g h = [x' \bmod q]$

- ▶ Problema PLD se poate rezolva, desigur, prin forță brută, calculând pe rând toate puterile x ale lui g până când se găsește una potrivită pentru care $g^x = h$;
- ▶ Complexitatea timp este $\mathcal{O}(q)$ iar complexitatea spațiu este $\mathcal{O}(1)$;
- ▶ Dacă se precalculează toate valorile (x, g^x) , căutarea se face în timp $\mathcal{O}(1)$ și spațiu $\mathcal{O}(q)$;
- ▶ Sunt de interes algoritmi care pot obține un timp mai bun la rulare decât forța brută, realizând un compromis spațiu-timp.

- ▶ Se cunosc mai mulți astfel de algoritmi împărțiți în două categorii:
 - ▶ algoritmi *generici* care funcționează în grupuri arbitrare (i.e. orice grupuri ciclice);
 - ▶ algoritmi *non-generici* care lucrează în grupuri *specifice* - exploatează proprietăți speciale ale anumitor grupuri
- ▶ Dintre algoritmi generici enumerăm:
- ▶ Metoda **Baby-step/giant-step**, datorată lui Shanks, calculează logaritmul discret într-un grup de ordin q în timp $\mathcal{O}(\sqrt{q} \cdot (\log q)^c)$;
- ▶ pentru $g \in \mathbb{G}$ generator, elementele lui $\mathbb{G}$ sunt

$$1 = g^0, g^1, g^2, \dots, g^{q-1}, g^q = 1$$

- ▶ știm că $h = g^x$ se află între aceste valori

Metoda Baby-step/giant-step

- marcam și memorăm anumite puncte din grup, aflate la distanța $t = \lfloor \sqrt{q} \rfloor$ (*giant-steps*)

$$g^0, g^t, g^{2t}, \dots, g^{\lfloor q/t \rfloor \cdot t}$$

- știm că $h = g^x$ se află în unul din aceste intervale
- calculand, cu *baby-steps*, valorile

$$h \cdot g^1, h \cdot g^2, \dots, h \cdot g^t$$

- una din ele va fi egala cu unul din punctele marcate i.e.
 $h \cdot g^i = g^{k \cdot t}$
- Complexitatea timp este $\mathcal{O}(\sqrt{q} \cdot \text{polylog}(q))$ iar complexitatea spațiu este $\mathcal{O}(\sqrt{q})$

► Metoda Baby-Step/Giant-Step este optima ca timp de rulare, insa exista alti algoritmi mai eficienti d.p.d.v. al complexitatii spatiu; ► Algoritmul Pohlig-Hellman poate fi folosit atunci cand se cunoaste factorizarea ordinului q al grupului iar timpul de rulare depinde de factorii primi ai lui q ; ► Pentru ca algoritmul sa nu fie eficient, trebuie ca cel mai mare factor prim al lui q sa fie de ordinul 2^{160} .

Algoritmi non-generici pentru calculul logaritmului discret

- Algoritmii non-generici sunt potențial mai puternici decât cei generici;
- Cel mai cunoscut algoritm pentru PLD în $\mathbb{Z}_p^*$ cu p prim este algoritmul GNFS (General Number Field Sieve) cu complexitate timp $2^{\mathcal{O}(n^{1/3} \cdot (\log n)^{2/3})}$ unde $|p| = \mathcal{O}(n)$;
- Există și un alt algoritm non-generic numit *metoda de calcul a indicelui* care rezolvă DLP în grupuri ciclice $\mathbb{Z}_p^*$ cu p prim în timp sub-exponențial în lungimea lui p .
- Această metodă seamănă cu algoritmul sitei pătratică pentru factorizare;

► Cel mai bun algoritm pentru DLP este sub-exponential; ► Se pot construi functii hash rezistente la coliziuni bazate pe dificultatea DLP

Schimbul de chei Diffie-Hellman

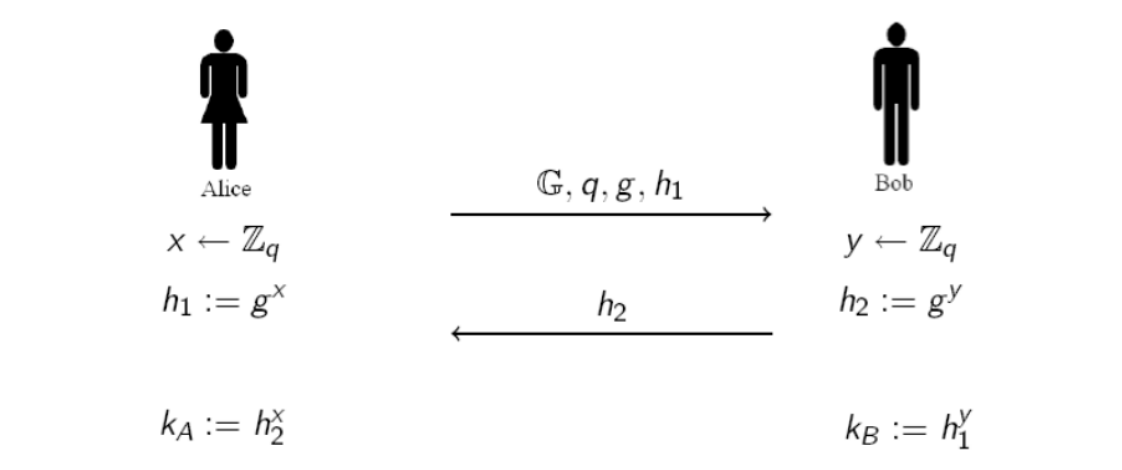
Primitive cu cheie publica ► Am vazut ca bazele criptografiei cu cheie publica au fost puse de Diffie ,si Hellman in 1976 ... ► ... c^and au introdus 3 primitive cu cheie publica diferite: 1. sisteme de criptare cu cheie publica 2. semnaturi digitale 3. schimb de chei ► Desi au introdus 3 concepte diferite, Diffie si Hellman au introdus o singura constructie, pentru schimbul de chei.

► **Sistemele de criptare cu cheie publica** le-am studiat si le vom mai studia in detaliu;

► **Semnaturile digitale** sunt analogul MAC-urilor din criptografia simetrica (sau corespondentul digital unei semnaturi reale);

► **Schimbul de chei** il introducem pentru a facilita introducerea sistemelor de criptare bazate pe DLP.
Schimb de chei

► Un protocol de schimb de chei este un protocol prin care 2 persoane care nu partajează în prealabil nici un secret pot genera o cheie comună, secretă; ► Comunicarea necesară pentru stabilirea cheii se realizează printr-un canal public! ► Deci un adversar poate intercepta toate mesajele transmise pe canalul de comunicație, dar NU trebuie să afle nimic despre cheia secretă obținută în urma protocolului; ► Protocoalele de schimb de chei reprezintă o primitivă fundamentală în criptografie; ► În continuare, ne vom rezuma strict la schimbul de chei Diffie-Hellman.



- Alice și Bob doresc să stabilească o cheie secretă comună;
- Alice generează un grup ciclic $\mathbb{G}$, de ordin q cu $|q| = n$ și g un generator al grupului;
- Alice alege $x \leftarrow^R \mathbb{Z}_q$ și calculează $h_1 := g^x$;
- Alice îi trimite lui Bob mesajul $(\mathbb{G}, g, q, h_1)$;
- Bob alege $y \leftarrow^R \mathbb{Z}_q$ și calculează $h_2 := g^y$;
- Bob îi trimite h_2 lui Alice și întoarce cheia $k_B := h_1^y$;
- Alice primește h_2 și întoarce cheia $k_A = h_2^x$.

- Corectitudinea protocolului presupune ca $k_A = k_B$, ceea ce se verifică ușor:

- Bob calculează cheia

$$k_B = h_1^y = (g^x)^y = g^{xy}$$

- Alice calculează cheia

$$k_A = h_2^x = (g^y)^x = g^{xy}$$

- ▶ O condiție **minimală** pentru ca protocolul să fie sigur este ca DLP să fie dificilă în $\mathbb{G}$;
- ▶ **Întrebare:** Cum poate un adversar pasiv să determine cheia comună dacă poate sparge DLP?
- ▶ **Răspuns:** Ascultă mediul de comunicație și preia mesajele h_1 și h_2 . Rezolvă DLP pentru h_1 și determină x , apoi calculează $k_A = k_B = h_2^x$.
- ▶ Aceasta nu este însă singura condiție necesară pentru a proteja protocolul de un atacator pasiv!

CDH (Computational Diffie-Hellman)

- ▶ O condiție mai potrivită ar fi că adversarul să nu poată determina cheia comună $k_A = k_B$, chiar dacă are acces la întreaga comunicație;
- ▶ Aceasta este **problema de calculabilitate Diffie-Hellman (CDH)**: Fiind date grupul ciclic $\mathbb{G}$, un generator g al său și 2 elemente $h_1, h_2 \xleftarrow{R} \mathbb{G}$, să se determine:

$$CDH(h_1, h_2) = g^{\log_g h_1 \log_g h_2}$$

- ▶ Pentru schimbul de chei Diffie-Hellman, rezolvarea CDH înseamnă că adversarul determină $k_A = k_B = g^{xy}$ cunoscând $h_1, h_2, \mathbb{G}, g$ (toate disponibile pe mediul de transmisiune nesecurizat).

DDH (Decisional Diffie-Hellman)

- ▶ Nici această condiție nu este suficientă: chiar dacă adversarul nu poate determina cheia exactă, poate de exemplu să determine părți din ea;
- ▶ O condiție și mai potrivită este ca pentru adversar, cheia $k_A = k_B$ să fie **indistinctibilă** față de o valoare aleatoare;
- ▶ Sau, altfel spus, să satisfacă **problema de decidabilitate Diffie-Hellman (DDH)**:

Definiție

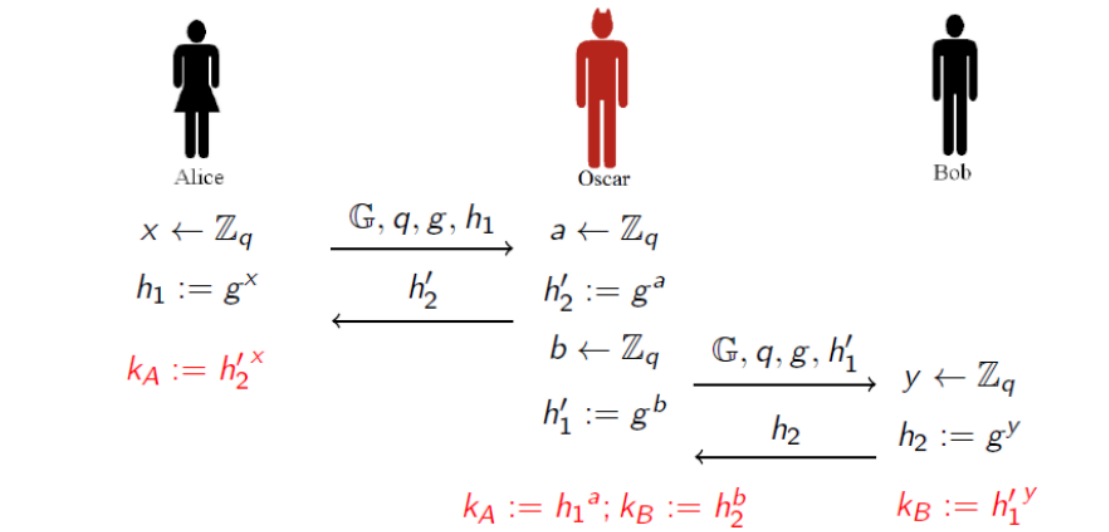
Spunem că problema decizională Diffie-Hellman (DDH) este dificilă (relativ la $\mathbb{G}$), dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât:

$$|Pr[\mathcal{A}(\mathbb{G}, q, g, g^x, g^y, g^z) = 1] - Pr[\mathcal{A}(\mathbb{G}, q, g, g^x, g^y, g^{xy}) = 1]| \leq \text{negl}(n),$$

unde $x, y, z \xleftarrow{R} \mathbb{Z}_q$

Atacul Man-in-the-Middle

► Am analizat pana acum securitatea fata de atacatori pasivi; ► Aratam acum ca schimbul de chei Diffie-Hellman este total nesigur pentru un adversar activ ... ► ... care are dreptul de a intercepta, modifica, elimina mesajele de pe calea de comunicatie; ► Un astfel de adversar se poate interpune intre Alice si Bob, dand nastere unui atac de tip Man-in-the-Middle.



- Alice generează un grup ciclic $\mathbb{G}$, de ordin q cu $|q| = n$ și g un generator al grupului;
- Alice alege $x \leftarrow^R \mathbb{Z}_q$ și calculează $h_1 := g^x$;
- Alice îi trimite lui Bob mesajul $(\mathbb{G}, g, q, h_1)$;
- Oscar interceptează mesajul și răspunde lui Alice în locul lui Bob: alege $a \leftarrow^R \mathbb{Z}_q$ și calculează $h'_2 := g^a$;
- Oscar și Alice dețin acum cheia comună $k_A = g^{xa}$;
- Oscar inițiază, în locul lui Alice, o nouă sesiune cu Bob: alege $b \leftarrow^R \mathbb{Z}_q$ și calculează $h'_1 := g^b$;
- Bob alege $y \leftarrow^R \mathbb{Z}_q$ și calculează $h_2 := g^y$;
- Oscar și Bob dețin acum cheia comună $k_B = g^{yb}$.

► Atacul este posibil pentru ca Oscar poate impersona pe Alice și pe Bob; ► De fiecare dată când Alice va transmite un mesaj criptat către Bob, Oscar îl interceptează și îl previne să ajungă la Bob; ► Oscar îl decriptează folosind k_A , apoi îl recriptează folosind k_B și îl transmite către Bob; ► Alice și Bob comunică fără să fie conștienți de existența lui Oscar

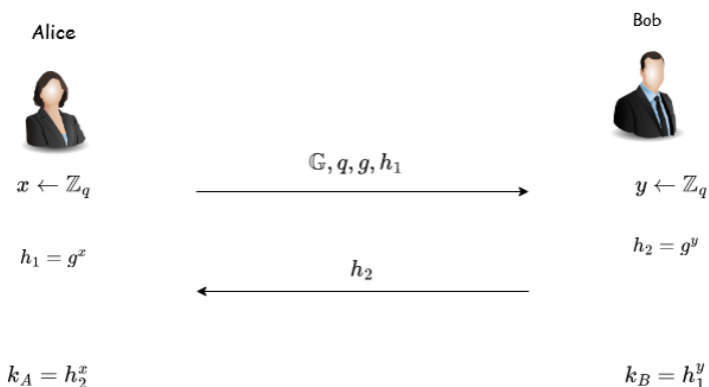
► **Schimbul de chei - o primitivă criptografică importantă** ► **Prezumții criptografice: CDH, DDH** ► **Schimbul de chei Diffie-Hellman nu rezistă la atacuri active**

Sistemul de criptare ElGamal

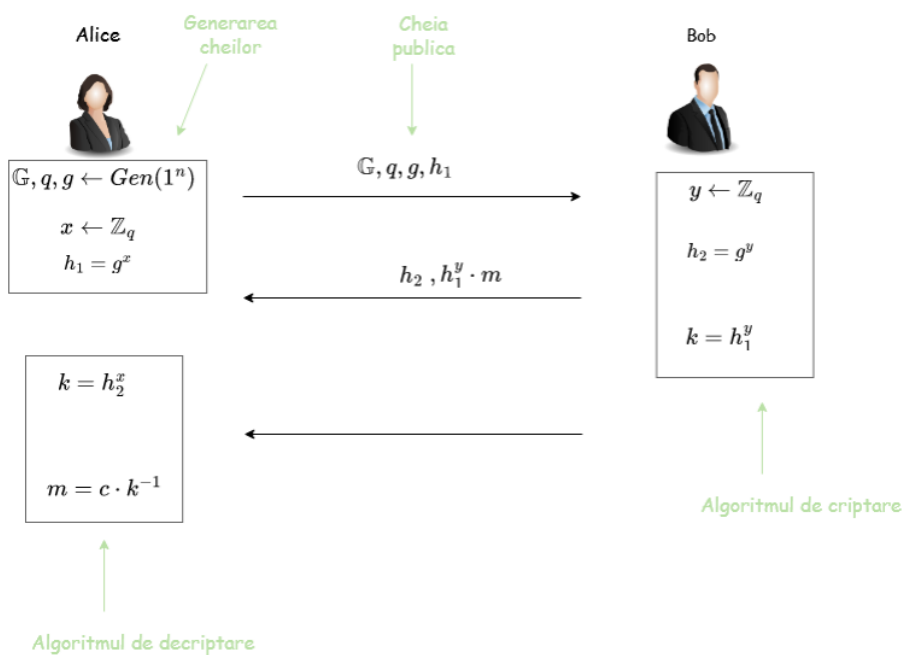
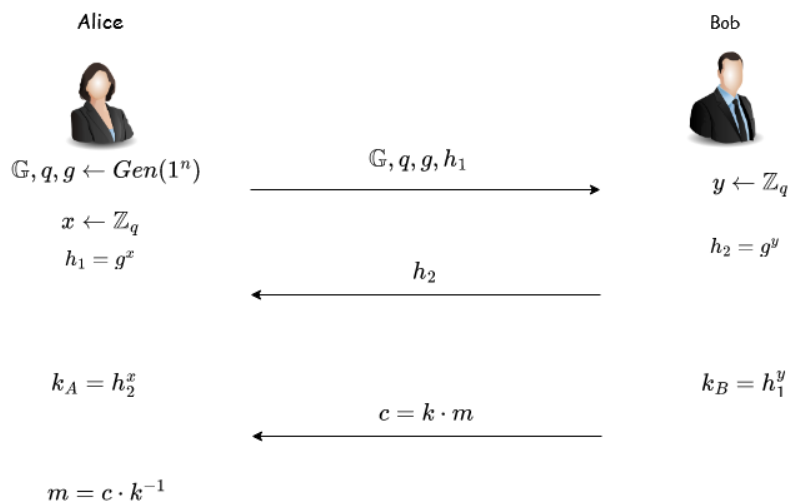
- 1976 - Diffie și Hellman definesc conceptul de criptografie asimetrică;
- 1977 - R.Rivest, A.Shamir și Leonard Adleman introduc sistemul RSA;
- 1985 - T.ElGamal propune un nou sistem de criptare.

Sistemul de criptare ElGamal

► Reamintim schimbul de chei Diffie-Hellman



► Il modificăm așa încât să permită criptare



- ▶ Definim sistemul de criptare *ElGamal* pe baza ideii prezentate anterior;
 1. Se generează $(\mathbb{G}, q, g)$, se alege $x \leftarrow^R \mathbb{Z}_q$ și se calculează $h = g^x$;
 - ▶ Cheia publică este: $(\mathbb{G}, q, g, h)$;
 - ▶ Cheia privată este x ;
 2. **Enc:** dată o cheie publică $(\mathbb{G}, q, g, h)$ și un mesaj $m \in \mathbb{G}$, alege $y \leftarrow^R \mathbb{Z}_q$ și întoarce $c = (c_1, c_2) = (g^y, m \cdot h^y)$;
 3. **Dec:** dată o cheie secretă $(\mathbb{G}, q, g, x)$ și un mesaj criptat $c = (c_1, c_2)$, întoarce $m = c_2 \cdot c_1^{-x}$.
-

Problema 1: Determinismul

- ▶ **Întrebare:** Este sistemul ElGamal determinist?
- ▶ **Răspuns:** NU! Sistemul este nedeterminist, datorită alegerii aleatoare a lui y la fiecare criptare.
- ▶ Un același mesaj m se poate cripta diferit, pentru $y \neq y'$:

$$c = (c_1, c_2) = (g^y, m \cdot h^y)$$

$$c' = (c'_1, c'_2) = (g^{y'}, m \cdot h^{y'})$$

Problema 2: Dificultatea DLP

- ▶ **Întrebare:** Rămâne ElGamal sigur dacă problema DLP este simplă?
 - ▶ **Răspuns:** NU! Se determină x a.î. $h = g^x$, apoi se decriptează orice mesaj pentru că se cunoaște cheia secretă.
-

Problema 3: Proprietatea de homomorfism

- ▶ Fie m_1, m_2 2 texte clare și $c_1 = (c_{11}, c_{12}), c_2 = (c_{21}, c_{22})$ textele criptate corespunzătoare;
 - ▶ Atunci:

$$c_1 \cdot c_2 = (c_{11} \cdot c_{21}, c_{12} \cdot c_{22}) = (g^{y_1} \cdot g^{y_2}, m_1 h^{y_1} \cdot m_2 h^{y_2})$$
 - ▶ **Întrebare:** Dacă un adversar cunoaște c_1 și c_2 criptările lui m_1 , respectiv m_2 , ce poate spune despre $c_1 \cdot c_2$?
 - ▶ **Răspuns:** $c_1 \cdot c_2$ este criptarea lui $m_1 \cdot m_2$ folosind $y = y_1 + y_2$:

$$c_1 \cdot c_2 = (g^{y_1+y_2}, m_1 m_2 h^{y_1+y_2})$$
 - ▶ Un sistem de criptare care satisface $Dec_{sk}(c_1 \cdot c_2) = Dec_{sk}(c_1) \cdot Dec_{sk}(c_2)$ se numește sistem de criptare **homomorfic**.
(homomorfismul este deseori o proprietate utilă în criptografie)
-

Problema 4: Utilizarea multiplă a parametrilor publici

- ▶ Este comun în practică pentru un administrator să fixeze parametrii publici $(\mathbb{G}, q, g)$, apoi fiecare utilizator să își genereze doar cheia secretă x și să publice $h = g^x$;
- ▶ **Întrebare:** Este corect să se utilizeze de mai multe ori aceiași parametrii publici $(\mathbb{G}, q, g)$?
- ▶ **Răspuns:** Se consideră că DA. Cunoașterea parametrilor publici pare să nu conducă la rezolvarea DDH.
- ▶ Atenție! Acest lucru nu se întâmplă și la RSA, unde modulul NU trebuie utilizat de mai multe ori.

Teoremă

Dacă problema decizională Diffie-Hellman (DDH) este dificilă în grupul $\mathbb{G}$, atunci schema de criptare ElGamal este CPA-sigură.

- ▶ Se poate vedea din securitatea schimbului de chei Diffie-Hellman
- ▶ În forma aceasta, sistemul ElGamal nu este CCA-sigur...pentru că este maleabil
Însă poate fi modificat așa încât să fie CCA-sigur

Criptografia bazată pe curbe eliptice

Definiție

O curbă eliptică peste $\mathbb{Z}_p$, $p > 3$ prim, este mulțimea perechilor (x, y) cu $x, y \in \mathbb{Z}_p$ așa încât

$$y^2 = x^3 + Ax + B \pmod{p}$$

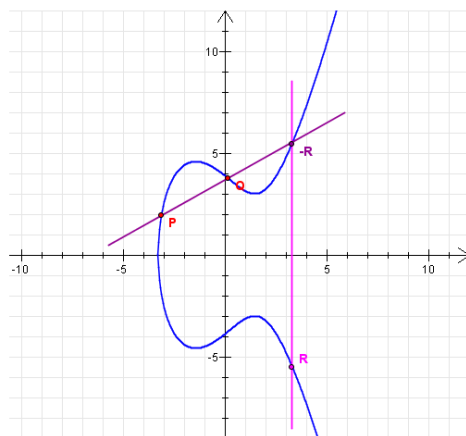
împreună cu punctul la infinit $\mathcal{O}$ unde

$A, B \in \mathbb{Z}_p$ sunt constante care respectă $4A^3 + 27B^2 \not\equiv 0 \pmod{p}$

- ▶ Vom nota cu $E(\mathbb{Z}_p)$ o curbă eliptică definită peste $\mathbb{Z}_p$

O curbă eliptică peste spațiul numerelor reale $\mathbb{R}$

$$E(\mathbb{R}) : y^2 = x^3 - x + 1$$



Grupul punctelor de pe o curbă eliptică

- Pentru a arăta că punctele de pe o curbă eliptică formează un grup ciclic, definim o operație de grup peste aceste puncte:

- Definim operația binară aditivă "+" astfel:

- punctul la infinit $\mathcal{O}$ este element neutru: $\forall P \in E(\mathbb{Z}_p)$ definim

$$P + \mathcal{O} = \mathcal{O} + P = P.$$

- fie $P = (x_1, y_1)$ și $Q = (x_2, y_2)$ două puncte de pe curbă; atunci:

- dacă $x_1 = x_2$ și $y_2 = -y_1$, $P + Q = \mathcal{O}$

- altfel, $P + Q = R$ de coordonate (x_3, y_3) care se calculează astfel:

$$\begin{aligned}x_3 &= [m^2 - x_1 - x_2 \bmod p] \\ y_3 &= [m(x_1 - x_3) - y_1 \bmod p]\end{aligned}$$

- iar m se calculează astfel:

$$m = \begin{cases} \frac{y_2 - y_1}{x_2 - x_1} \bmod p & \text{dacă } P \neq Q \\ \frac{3x_1^2 + A}{2y_1} \bmod p & \text{dacă } P = Q \end{cases}$$

- dacă $P = Q$ și $y_1 = 0$, atunci $P + Q = 2P = \mathcal{O}$

- În practică, sunt căutate acele curbe eliptice pentru care ordinul grupului ciclic generat este prim;
- Pentru criptografie, sunt de interes curbe eliptice de ordin mare
- O curbă eliptică definită peste $\mathbb{Z}_p$ are aproximativ p puncte. Mai precis [Teorema lui Hasse]:

$$p + 1 - 2\sqrt{p} \leq \text{card}(E(\mathbb{Z}_p)) \leq p + 1 + 2\sqrt{p}$$

- Există mai multe clase de curbe eliptice slabe d.p.d.v. criptografic, iar ele trebuie evitate.
- De pildă, curbe eliptice peste $\mathbb{Z}_p$ cu $\text{card}(E(\mathbb{Z}_p)) = p$
- În practică, se folosesc curbe eliptice standardizate

Curbe eliptice folosite în practică

Curbe eliptice standardizate, folosite în practică, sigure și cu implementări eficiente:

- ▶ *curba eliptică P-256* (sau *secp256r1*) este o curbă eliptică peste $\mathbb{Z}_p$ cu p pe 256 biți de forma $p = 2^{256} - 2^{224} + 2^{192} + 2^{96} - 1$. Această curbă are ecuația $y^2 = x^3 - 3x + B \pmod{p}$ iar p -ul ales astfel permite o implementare eficientă. Curbele *P-384* (*secp384r1*) și *P-521* (*secp521r1*) sunt definite în mod analog
- ▶ *curba eliptică 25519* este definită peste $\mathbb{Z}_p$ cu p pe 255 biți de forma $p = 2^{255} - 19$ și permite o implementare eficientă. Grupul acestei curbe eliptice nu are ordin prim dar se poate lucra într-un subgrup de ordin mare prim

Curbe eliptice standardizate, folosite în practică, sigure și cu implementări eficiente:

- ▶ *secp256k1* este o curbă eliptică de ordin prim peste $\mathbb{Z}_p$ cu p pe 256 biți de forma $p = 2^{256} - 2^{232} - 2^{29} - 2^{28} - 2^{27} - 2^{26} - 2^{24} - 1$ și are ecuația $y^2 = x^3 + 7 \pmod{p}$. Aceasta curbă eliptică este folosită în Bitcoin.

ECDLP - Problema logaritmului discret pe curbe eliptice

- ▶ **ECDLP** = **E**lliptic **C**urve **D**iscrete **L**ogarithm **P**roblem
- ▶ Putem defini acum DLP în grupul punctelor unei curbe eliptice (ECDLP):
- ▶ Fie E o curbă eliptică peste $\mathbb{Z}_p$, un punct $P \in \mathbb{Z}_p$ de ordin n și Q un element din subgrupul ciclic generat de P :

$$Q \in [P] = \{sP \mid 1 \leq s \leq n-1\}$$

- ▶ Problema ECDLP cere găsirea unui k așa încât $Q = kP$;
- ▶ Notăție: $\underbrace{P + P + \dots + P}_{s \text{ ori}} = sP$.

- ▶ Alegând cu grijă curbele eliptice, cel mai bun algoritm pentru rezolvarea ECDLP este considerabil mai slab decât cel mai bun algoritm pentru rezolvarea problemei DLP în $\mathbb{Z}_p^*$;
- ▶ De exemplu, algoritmul de calcul al indicelui nu este deloc eficient pentru ECDLP;
- ▶ Pentru anumite curbe eliptice, singurii algoritmi de rezolvare sunt algoritmi generici pentru DLP, adică metoda baby-step giant-step și metoda Pollard rho;
- ▶ Cum numărul de pași necesari pentru un astfel de algoritm este de ordinul rădăcinii păturate a cardinalului grupului, se recomandă folosirea unui grup de ordin cel puțin 2^{160} .
- ▶ O consecință a teoremei lui Hasse este că dacă avem nevoie de o curbă eliptică cu 2^{160} elemente, trebuie să folosim un număr prim p pe aproximativ 160 biți;
- ▶ Deci, dacă folosim o curbă eliptică $E(\mathbb{Z}_p)$ cu p pe 160 biți, un atac generic asupra ECDLP are 2^{80} complexitate timp;
- ▶ Un nivel de securitate de 80 biți oferă securitate pe termen mediu;
- ▶ În practică, curbe eliptice peste $\mathbb{Z}_p$ cu p până la 256 biți sunt folosite, cu un nivel de securitate pe 128 biți.

Comparație între ECC, criptografia simetrică și asimetrică

Algorithm Family	Cryptosystems	Security Level (bit)			
		80	128	192	256
Integer factorization	RSA	1024 bit	3072 bit	7680 bit	15360 bit
Discrete logarithm	DH, DSA, Elgamal	1024 bit	3072 bit	7680 bit	15360 bit
Elliptic curves	ECDH, ECDSA	160 bit	256 bit	384 bit	512 bit
Symmetric-key	AES, 3DES	80 bit	128 bit	192 bit	256 bit

Figure: [Understanding cryptography, Christoph Paar, Jan Pelzl, Springer 2010]

- ▶ Un algoritm are nivelul de securitate "security level" pe n biți dacă cel mai bun atac necesită 2^n pași.
- ▶ Curbele eliptice ofera un suport bun pentru criptografie; ▶ ECDLP este dificila.

Schimbul de chei Diffie-Hellman pe curbe eliptice

- ▶ Alice și Bob doresc să stabilească o cheie secretă comună;
- ▶ Alice generează o curbă eliptică $E(\mathbb{Z}_q)$, și P un punct pe curbă (generator);
- ▶ Alice alege $x \leftarrow^R \mathbb{Z}_q$ și calculează $H_1 := xP$;
- ▶ Alice îi trimite lui Bob mesajul $(E(\mathbb{Z}_q), P, H_1)$;
- ▶ Bob alege $y \leftarrow^R \mathbb{Z}_q$ și calculează $H_2 := yP$;
- ▶ Bob îi trimite H_2 lui Alice și întoarce cheia $k_B := yH_1$;
- ▶ Alice primește H_2 și întoarce cheia $k_A = xH_2$.
- ▶ Corectitudinea protocolului presupune ca $k_A = k_B$, ceea ce se verifică ușor:
- ▶ Bob calculează cheia

$$k_B = yH_1 = y(xP) = (xy)P$$

- ▶ Alice calculează cheia

$$k_A = xH_2 = x(yP) = (xy)P$$

- ▶ O condiție **minimală** pentru ca protocolul să fie sigur este ca ECDLP să fie dificilă în $\mathbb{G}$;
- ▶ **Întrebare:** Cum poate un adversar pasiv să determine cheia comună dacă poate sparge ECDLP?
- ▶ **Răspuns:** Ascultă mediul de comunicație și preia mesajele H_1 și H_2 . Rezolvă ECDLP pentru H_1 și determină x , apoi calculează $k_A = k_B = xH_2$.
- ▶ Aceasta nu este însă singura condiție necesară pentru a proteja protocolul de un atacator pasiv!

ECDDH (Elliptic Curve Computational Diffie-Hellman)

- ▶ O condiție mai potrivită ar fi că adversarul să nu poată determina cheia comună $k_A = k_B$, chiar dacă are acces la întreaga comunicație;
- ▶ Aceasta este **problema de calculabilitate Diffie-Hellman pe curbe eliptice (ECDDH)**: Fiind date curba eliptică $E(\mathbb{Z}_q)$, un punct P pe curbă și 2 alte puncte $H_1, H_2 \leftarrow^R E(\mathbb{Z}_q)$, să se determine:

$$ECDDH(H_1, H_2) = (ECDLP(P, H_1)ECDLP(P, H_2))P$$

- ▶ Pentru schimbul de chei Diffie-Hellman, rezolvarea ECDDH înseamnă că adversarul determină $k_A = k_B = xyP$ cunoscând $H_1, H_2, E(\mathbb{Z}_q), P$ (toate disponibile pe mediul de transmisiune nesecurizat).

- Nici această condiție nu este suficientă: chiar dacă adversarul nu poate determina cheia exactă, poate de exemplu să determine părți din ea;
- O condiție și mai potrivită este ca pentru adversar, cheia $k_A = k_B$ să fie **indistinctibilă** față de o valoare aleatoare;
- Sau, altfel spus, să satisfacă **problema de decidabilitate Diffie-Hellman pe curbe eliptice (ECDDH)**:

Definiție

Spunem că problema decizională Diffie-Hellman (ECDDH) este dificilă (relativ la curba eliptică $E(\mathbb{Z}_q)$), dacă pentru orice algoritm PPT $\mathcal{A}$ există o funcție neglijabilă negl așa încât:

$$|\Pr[\mathcal{A}(E(\mathbb{Z}_q), P, xP, yP, zP) = 1] - \Pr[\mathcal{A}(E(\mathbb{Z}_q), P, xP, yP, xyP) = 1]| \leq \text{negl}(n), \text{ unde } x, y, z \leftarrow^R \mathbb{Z}_q$$

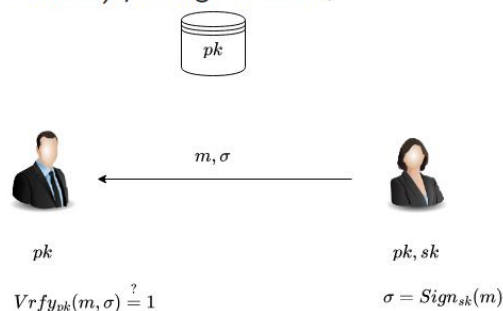
Important de reținut!

- Modalitatea de trecere de la o construcție peste $(\mathbb{Z}_q, \cdot)$ la $(E(\mathbb{Z}_q), +)$
- Prezumții criptografice: ECCDH, ECDDH
- Schimbul de chei Diffie-Hellman pe curbe eliptice păstrează proprietățile schimbului de chei Diffie Hellman definit peste $\mathbb{G}$ grup ciclic de ordin q

Curs 11 - Semnături digitale și PKI

Scheme de semnătură digitală

- Schemele de semnătură digitală reprezintă echivalentul MAC-urilor în criptografia cu cheie publică, deși există câteva diferențe importante între ele;
- O schemă de semnătură digitală îi permite unui semnatar S care a stabilit o cheie publică pk să semneze un mesaj în așa fel încât oricine care cunoaște cheia pk poate verifica originea mesajului (ca fiind S) și integritatea lui;



Aplicații ale semnăturilor digitale ► De pilda, o companie de software vrea să transmită patch-uri de software într-o manieră autenticată așa încât orice client să poată recunoaște dacă un patch este autentic; ► În schimb, o persoană malicioasă nu poate păcăli un client să accepte un patch care nu a fost realizat de compania respectivă.

► Pentru aceasta, compania generează o cheie publică pk împreună cu o cheie secretă sk și distribuie cheia pk clientilor săi, păstrând cheia secretă; ► Atunci când lansează un patch de software, compania calculează o semnătură digitală σ pentru patch folosind cheia sk și trimite fiecărui client perechea (patch, σ); ► Fiecare client stabilește autenticitatea lui patch verificând dacă σ este o semnătură legitimă pentru patch cu privire la cheia publică pk ; ► Deci compania folosește aceeași cheie publică pentru toți clienții și calculează o singură semnătură pe care o trimite tuturor

Avantaje semnături digitale față de MAC-uri

► MAC-urile și schemele de semnătură digitală sunt folosite pentru asigurarea integrității (autenticității) mesajelor cu următoarele **diferențe**: ► Schemele de semnătură digitală sunt public verificabile... ► ...ceea ce înseamnă că semnăturile digitale sunt transferabile - o terță parte poate verifica legitimitatea unei semnături și poate face o copie pentru a convinge pe altcineva că aceea este o semnătură validă pentru m ; ► Schemele de semnătură digitală au **proprietatea de non-repudiare** - un semnatar nu poate nega faptul că a semnat un mesaj; ► MAC-urile au avantajul că sunt cam de 2-3 ori mai eficiente (mai rapide) decât schemele de semnătură digitală.

Definiție

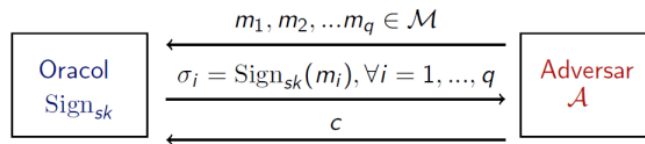
O semnătură digitală definită peste $(\mathcal{K}, \mathcal{M}, \mathcal{S})$ este formată din trei algoritmi polinomiali (Gen, Sign, Vrfy) unde:

1. Gen: este algoritmul de generare a perechii de cheie publică și cheie privată (pk, sk)
2. Sign : $\mathcal{K} \times \mathcal{M} \rightarrow \mathcal{S}$ este algoritmul de generare a semnăturilor $\sigma \leftarrow \text{Sign}_{sk}(m)$;
3. Vrfy : $\mathcal{K} \times \mathcal{M} \times \mathcal{S} \rightarrow \{0, 1\}$ este algoritmul de verificare ce întoarce un bit $b = \text{Vrfy}_{pk}(m, \sigma)$ cu semnificația că:
 - $b = 1$ înseamnă valid
 - $b = 0$ înseamnă invalid

a.î : $\forall m \in \mathcal{M}, k \in \mathcal{K}, \text{Vrfy}_{pk}(m, \text{Sign}_{sk}(m)) = 1.$

- Formal, îi dăm adversarului acces la un oracol $\text{Sign}_{sk}(\cdot)$;
- Adversarul poate trimite orice mesaj m dorit către oracol și primește înapoi o semnătură corespunzătoare $\sigma \leftarrow \text{Sign}_{sk}(m)$;
- Considerăm că securitatea este impactată dacă adversarul este capabil să producă un mesaj m împreună cu o semnătură σ așa încât:
 1. σ este o semnătură validă pentru mesajul m :
 $\text{Vrfy}_{pk}(m, \sigma) = 1$;
 2. Adversarul nu a solicitat anterior (de la oracol) o semnătură pentru mesajul m .

Securitate semnatura digitală - formalizare ► Despre o semnătură care satisface nivelul de securitate de mai sus spunem că nu poate fi falsificată printr-un atac cu mesaj ales; ► Aceasta înseamnă că un adversar nu este capabil să falsifice o semnătură validă pentru nici un mesaj ... ► ... deși poate obține semnături pentru orice mesaj ales de el, chiar adaptiv în timpul atacului. ► Pentru a da definiția formală, definim mai întâi un experiment pentru o semnătură $\pi = (\text{Sign}, \text{Vrfy})$, în care considerăm un adversar A și parametrul de securitate n ;



- ▶ Adversarul întoarce o pereche de mesaj, semnătură (m, σ)
- ▶ Output-ul experimentului este 1 dacă și numai dacă:
(1) $\text{Vrfy}_{pk}(m, \sigma) = 1$ și (2) $m \notin \{m_1, \dots, m_q\}$;
- ▶ Dacă $\text{Sign}_{\mathcal{A}, \pi}^{\text{forge}}(n) = 1$, spunem că $\mathcal{A}$ a efectuat experimentul cu succes.

Semnatura digitală bazată pe RSA

- ▶ Gen: generează N, e, d și $pk = (N, e)$ iar $sk = d$
- ▶ $\text{Sign}(sk, m)$: semnează mesajul m folosind cheia $sk = d$ astfel

$$\sigma = m^d \bmod N$$

- ▶ $\text{Vrfy}(pk, m, \sigma)$: semnătura este validă dacă și numai dacă

$$m \stackrel{?}{=} \sigma^e \bmod N$$

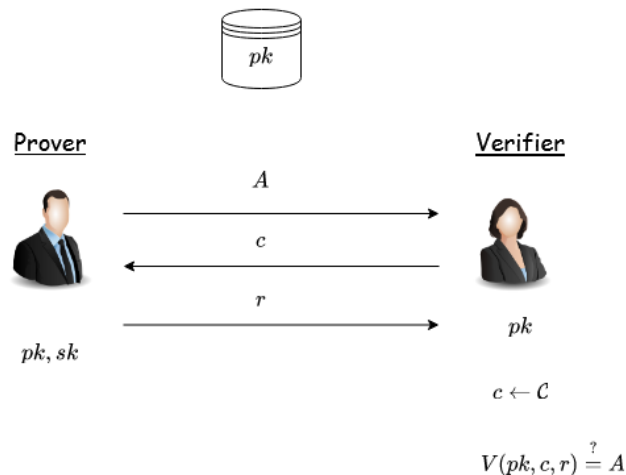
Această variantă de semnătură nu este sigură, se cunosc mai multe atacuri pentru ea

- ▶ scop adversar: falsificarea semnăturii mesajului $m \in \mathbb{Z}_N^*$ pentru $pk = (N, e)$
- ▶ acțiune adversar: alege $m_1, m_2 \in \mathbb{Z}_N^*$ a.i. $m = m_1 \cdot m_2 \bmod N$
- ▶ obține semnăturile σ_1 și σ_2 pentru mesajele m_1, m_2
- ▶ întoarce $\sigma = \sigma_1 \cdot \sigma_2 \bmod N$ ca semnătură validă pentru m
- ▶ aceasta este o semnătură validă pentru că

$$\sigma^e = (\sigma_1 \cdot \sigma_2)^e = (m_1^d \cdot m_2^d)^e = m_1^{ed} \cdot m_2^{ed} = m_1 \cdot m_2 = m \bmod N$$

Scheme de identificare - identification schemes

- ▶ sunt protocoale interactive care permit unei părți (Prover) să își demonstreze identitatea în fața unei alte părți (Verifier)
- ▶ sunt foarte importante ca building block pentru semnături digitale (dar, în sine, au aplicabilitate limitată)
- ▶ în continuare, abordare informală



▶ Securitate ▶ fata de adversarii pasivi - chiar daca are acces la mesajele trimise in mai multe executii ale protocolului, un adversar nu il poate convinge pe Verifier sa accepte ▶ Principala aplicatie ▶ identificarea persoanelor prezente fizic; de pilda, deschiderea unei usi securizate pe baza unei cartele de acces ▶ nu este potrivita pentru autentificarea la distanta (pe internet)

Certificate și PKI

- ▶ O problemă a criptografiei cu cheie publică o reprezintă distribuirea cheilor publice;
- ▶ Se rezolvă tot cu criptografia cu cheie publică: e suficient să distribuim o singură cheie publică în mod sigur...
- ▶ Ulterior ea poate fi folosită pentru a distribui sigur oricât de multe chei publice;
- ▶ Ideea constă în folosirea unui *certificat digital* care este o semnătură care atașează unei entități o anume cheie publică;

- ▶ De exemplu, dacă Charlie are cheia generată (pk_C, sk_C) iar Bob are cheia (pk_B, sk_B) , iar Charlie cunoaște pk_B atunci el poate calcula semnătura de mai jos pe care i-o dă lui Bob:

$$\text{cert}_{C \rightarrow B} = \text{Sign}_{sk_C}(\text{"Cheia lui Bob este } pk_B\text{"})$$

- ▶ Această semnătură este un *certificat* emis de Charlie pentru Bob;
- ▶ Atunci când Bob vrea să comunice cu Alice, îi trimite întâi cheia publică pk_B împreună cu certificatul $\text{cert}_{C \rightarrow B}$ a cărui validitate în raport cu pk_C Alice o verifică;

Curs 12- Criptografia post-cuantica

Criptografia post-cuantica ► In cadrul criptografiei de pana acum am discutat despre un adversar PPT care ruleaza in timp polinomial pe un calculator conventional (clasic). In evaluarea securitatii primitivelor criptografie am considerat numai atacuri clasice. ► Nu am avut in vedere calculatoarele cuantice - care se bazeaza pe principiile mecanicii cuantice si impactul lor asupra securitatii ► Algoritmi cuantici pot fi, in anumite cazuri, mult mai rapizi decat cei clasici si pot avea un impact zdrobitor asupra securitatii primitivelor criptografice studiate.

► D.p.d.v. teoretic, impactul calculatoarelor cuantice asupra criptografiei este recunoscut din anii 1990. ► Practic, un calculator cuantic generic, pe scara larga nu exista iar costurile pentru constructia lui ar fi uriase ► Totusi, un calculator cuantic dedicat care sa atace sistemele de criptare actuale ar putea aparea in decurs de cativa ani sau zeci de ani. ► Odata ce un astfel de calculator cuantic care sa poate fi folosit in practica devine disponibil, toti algoritmi cu cheie publica folositi in prezent dar si protocoalele asociate devin vulnerabile ► Aceasta inseamna ca toate email-urile, informatiile despre cardurile cu care facem plati online, semnături digitale, tranzactii online, datele sensibile, informatiile clasificate ale agentilor de securitate si cele guvernamentale vor fi in pericol

Competiția NIST pentru standardizare post-cuantică

- Iulie 2022 - NIST anunță primul grup de 4 finaliști
 - **criptare**: CRYSTALS - Kyber - pentru chei mici de criptare și operații rapide
 - **semnături digitale**
 - CRYSTALS-Dilithium
 - FALCON
 - SPHINCS+
- Dilithium și Falcon sunt foarte eficienți, cel din urmă fiind recomandat atunci când sunt necesare semnături mai mici decât cele oferite de Dilithium
- Primii 3 algoritmi se bazează pe probleme matematice de latici, iar SPHINCS+ folosește funcții hash
- Procesul de standardizare se va finaliza în anul 2024
- Alți 4 algoritmi sunt considerați pentru standardizare

Criptografia cuantică

- implementări folosind calculatoare cuantice, fenomene mecanice cuantice și canale de comunicare cuantice
- dificil de implementat la scara largă
- în unele cazuri este sigură necondiționat (nu se bazează pe ipoteze matematice)

Criptografia post-cuantică

- implementări folosind calculatoare clasice
- este sigură chiar și în fața unui adversar care are acces la un calculator cuantic
- se bazează pe probleme matematice dificile computațional chiar și pentru algoritmi cuantici

- Impactul calculatoarelor cuantice asupra criptografiei simetrice este minor; ilustrăm pe scurt
- Considerăm următoarea **problema abstractă**:
 - Se dă: funcție $f : D \rightarrow \{0, 1\}$ cu acces de tip oracol (funcția poate fi interogată pe orice input și se primește output-ul corespunzător)
 - Se cere: să se găsească x a.î. $f(x) = 1$.
- Dacă există un singur x cu $f(x) = 1$ atunci orice algoritm clasic necesită $O(\|D\|)$ evaluări ale funcției f - corespunde unui atac prin forță brută
- 1996 - **algoritmul cuantic al lui Grover**: găsește x folosind $O(\|D\|^{1/2})$ evaluări ale funcției f . Algoritmul este optim, și nu poate fi îmbunătățit

Criptografia simetrică post-cuantică - sisteme bloc

- Trecem în revistă impactul algoritmului asupra sistemelor de criptare simetrice
- Considerăm cazul unui sistem de criptare bloc
 $F : \{0, 1\}^n \times \{0, 1\}^l \rightarrow \{0, 1\}^l$ cu cheia k pe n biți pentru care cel mai bun atac (de găsire a cheii) este forța brută.
- Un astfel de atac clasic necesită timp 2^n
- Pentru securitate, alegem cheia k de lungime n biți așa încât timpul pentru atac 2^n să nu fie practic
- Algoritmul lui Grover însă permite unui atacator să găsească cheia în timp $2^{n/2}$.
- Pentru același nivel de securitate (precum în cazul clasic), alegem cheia k de lungime **dublă** față de cazul clasic.

Criptografia simetrică post-cuantică - funcții hash

- Considerăm problema găsirii de coliziuni pentru o funcție hash
 $H : \{0, 1\}^m \rightarrow \{0, 1\}^n$ cu $m > n$.
- În cazul **clasic**, am văzut că "atacul nașterilor" necesită $O(2^{n/2})$ evaluări ale funcției H .
- Aceasta înseamnă că pentru a asigura rezistența la coliziuni față de un atac în timp 2^t , trebuie să alegem funcții hash cu **output-ul pe $2t$ biți**.
- Însă în cazul **cuantic**, un atac pentru găsirea coliziunilor necesită $O(2^{n/3})$ evaluări ale funcției H .
- Deci, pentru a asigura rezistența la coliziuni față de un atac în timp 2^t , trebuie să alegem funcții hash cu **output-ul pe $3t$ biți**.

Algoritmul lui Shor si impactul lui asupra criptografiei asimetrice

► Pana acum am vazut algoritmi cuantici care ofera o imbunatatire de ordin polinomial in comparatie cu cei mai buni algoritmi clasici pentru aceeasi problema. ► Acestia impun doar cresterea dimensiunilor cheilor fara a necesita alte schimbari majore ► In continuare vom vedea un algoritm care ofera o imbunatatire de ordin exponential - **algoritmi cuantici polinomiali pentru problema factorizarii si problema logaritmului discret**

► Algoritmul lui Shor poate fi folosit si pentru rezolvarea problemei logaritmului discret in timp polinomial ► Avand in vedere ca toate sistemele de criptare cu cheie publica se bazeaza pe problema factorizarii sau problema logaritmului discret, concluzionam ca

Toate sistemele de criptare cu cheie publica prezentate la curs pot fi atacate in timp polinomial cu ajutorul unui calculator cuantic

Sisteme de criptare cu cheie publica post-cuantice

► Problema factorizarii si problema logaritmului discret devin "usoare" pentru un calculator cuantic.

► Avem nevoie de probleme matematice dificile computational chiar si pentru calculatoarele cuantice, dar care ruleaza pe calculatoare clasice ► Diferenta fata de cazul clasic este ca problemele considerate pentru criptografia post-cuantica sunt mai recente si nu au fost studiate la fel de mult ca problema factorizarii sau problema logaritmului discret ► In continuare vom prezenta o problema care a primit multa atentie si care este considerata dificila chiar si pentru calculatoarele cuantice. Aratam apoi cum se poate construi un sistem de criptare cu cheie publica bazat pe dificultatea acelei probleme.

Problema LWE - Learning With Errors

► A fost introdusa in 2005 de Oded Regev

► Preliminarii:

- q număr prim. Vom nota cu $\mathbb{Z}_q$ mulțimea $\{-\lfloor (q-1)/2 \rfloor, \dots, 0, \dots, \lfloor q/2 \rfloor\}$ (spre deosebire de $\{0, \dots, q\}$) unde $\lfloor x \rfloor$ este cel mai mare intreg mai mic sau egal cu x .
- spunem că un element al lui $\mathbb{Z}_q$ este "mic" dacă este "aproape" de 0.

► Problema cere găsirea lui $\mathbf{s} \in \mathbb{Z}_q^n$ fiind dată o secvență de ecuații liniare "aproximative" în $\mathbf{s}$.

► Exemplu:

$$12s_1 + 10s_2 + 5s_3 + 2s_4 \approx 8 \bmod 17$$

$$3s_1 + 7s_2 + 9s_3 + s_4 \approx 4 \bmod 17$$

$$16s_1 + 2s_2 + 8s_3 + 7s_4 \approx 3 \bmod 17$$

► Exemplul

$$12s_1 + 10s_2 + 5s_3 + 2s_4 + 1 = 8 \bmod 17$$

$$3s_1 + 7s_2 + 9s_3 + s_4 - 1 = 4 \bmod 17$$

$$16s_1 + 2s_2 + 8s_3 + 7s_4 + 2 = 3 \bmod 17$$

sub forma matriciala

<table border="1"><tr><td>12</td><td>10</td><td>5</td><td>2</td></tr><tr><td>3</td><td>7</td><td>9</td><td>1</td></tr><tr><td>16</td><td>2</td><td>8</td><td>7</td></tr></table>	12	10	5	2	3	7	9	1	16	2	8	7	*	<table border="1"><tr><td>s_1</td></tr><tr><td>s_2</td></tr><tr><td>s_3</td></tr><tr><td>s_4</td></tr></table>	s_1	s_2	s_3	s_4	+	<table border="1"><tr><td>1</td></tr><tr><td>-1</td></tr><tr><td>2</td></tr></table>	1	-1	2	≈	<table border="1"><tr><td>8</td></tr><tr><td>4</td></tr><tr><td>3</td></tr></table>	8	4	3
12	10	5	2																									
3	7	9	1																									
16	2	8	7																									
s_1																												
s_2																												
s_3																												
s_4																												
1																												
-1																												
2																												
8																												
4																												
3																												

sau, notand matricile corespunzator (unde $\mathbf{s} = (s_1, s_2, s_3, s_4)$), ecuația matricială devine

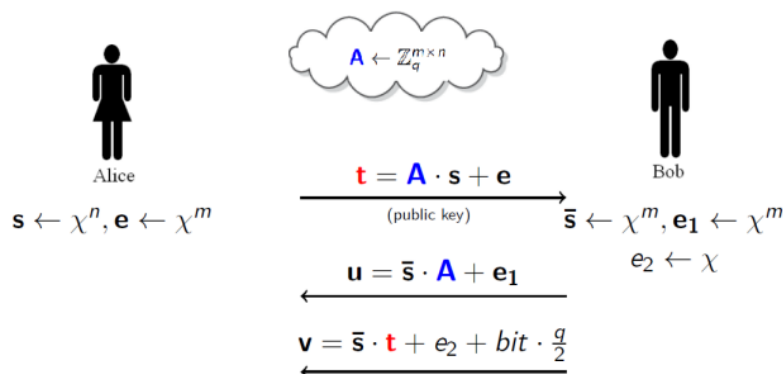
$$\mathbf{A}\mathbf{s} + \mathbf{e} = \mathbf{b} \bmod q$$

$$\begin{bmatrix} 12 & 10 & 5 & 2 \\ 3 & 7 & 9 & 1 \\ 16 & 2 & 8 & 7 \end{bmatrix} * \begin{bmatrix} s_1 \\ s_2 \\ s_3 \\ s_4 \end{bmatrix} + \begin{bmatrix} 1 \\ -1 \\ 2 \end{bmatrix} \approx \begin{bmatrix} 8 \\ 4 \\ 3 \end{bmatrix}$$

- ▶ vectorul $\mathbf{e} = (1, -1, 2)$ este format din elemente "mici" din $\mathbb{Z}$ numite *noise* sau *error*.
- ▶ In lipsa lui $\mathbf{e}$, ecuația $\mathbf{As} = \mathbf{b}$ devine usor de rezolvat cu tehnici clasice de algebra liniară
- ▶ Cand matricea $\mathbf{A}$ are suficient de multe linii și parametrii sunt aleși corespunzător, problema devine dificilă.

Sistem de criptare bazat pe LWE

- ▶ Protocolul de mai sus nu este sigur pentru că un atacator poate calcula $\bar{s}$ sau $\mathbf{s}$ cu noțiuni de algebră liniară și poate afla și cheia.
- ▶ Însă el poate fi transformat într-un protocol sigur și adaptat ca un sistem de criptare adăugând "noise", sub ipoteza problemei decizionale LWE.



- ▶ Schema de mai sus criptează un bit iar decriptarea se face calculând $x = \mathbf{v} - \mathbf{u} \cdot \mathbf{s}$.
- ▶ Rezultatul va fi 1 dacă x este mai aproape de $\frac{q}{2}$ decât de 0.
- ▶ Apropierea lui x de $\frac{q}{2}$ este verificată calculând valoarea absolută a lui $x - \frac{q}{2} \bmod q$

Exerciții

Fie El Gamal cu $pk = (g, h = g^a)$ și $sk = (g, a)$ în $\mathbb{G}$.

- **Enc:** dată o cheie publică $(\mathbb{G}, q, g, h)$ și un mesaj $m \in \mathbb{G}$, alege $y \leftarrow^R \mathbb{Z}_q$ și întoarce $c = (c_1, c_2) = (g^y, m \cdot h^y)$;
- **Dec:** dată o cheie secretă $(\mathbb{G}, q, g, a)$ și un mesaj criptat $c = (c_1, c_2)$, întoarce $m = c_2 \cdot c_1^{-a}$.

Vrem să distribuim cheia secretă la două persoane așa încât numai cele două persoane împreună pot decripta. O modalitate simplă de a rezolva această problemă este să alegem două numere aleatoare $a_1, a_2 \in \mathbb{Z}_n$ așa încât $a_1 + a_2 = a$. O persoană primește a_1 iar cealaltă primește a_2 . Pentru decriptarea (c_1, c_2) , trimitem c_1 ambelor persoane.

Ce valori trebuie să calculeze cele două persoane și să ne trimită înapoi așa încât să putem decripta textul criptat trimis?

Solution

Persoana 1 trimite $u_1 = c_1^{a_1}$ iar persoana 2 trimite $u_2 = c_1^{a_2}$.

Produsul $u_1 \cdot u_2 = c_1^{a_1+a_2} = c_1^a$ împreună cu c_2 poate fi folosit la

Securitate în Sistemelor Informatic

24 / 26

Se consideră $H : \{0, 1\}^* \rightarrow \{0, 1\}^n$ o funcție hash rezistentă la a doua preimagine și rezistentă la coliziuni. Se definește o funcție $H^* : \{0, 1\}^* \rightarrow \{0, 1\}^{n+1}$ astfel:

$$H'(x) = \begin{cases} x||1 & \text{dacă } x \in \{0, 1\}^n \\ H(x)||0 & \text{altfel} \end{cases}$$

Argumentați că H' este rezistentă la coliziuni.

Solution

Fie $H'(x_1) = H'(x_2)$ cu $x_1 \neq x_2$. Dacă $H'(x_1) = x||1$ rezultă $x_1 = x_2$, contradicție. Dacă $H'(x_1) = H(x_1)||0 = H(x_2)||0$ atunci se determină o coliziune pentru H , contradicție.

Fie $(\text{Mac}, \text{Vrfy})$ un MAC sigur definit peste (K, M, T) unde $M = \{0, 1\}^n$ și $T = \{0, 1\}^{128}$. Este MAC-ul de mai jos sigur? Argumentați răspunsul.

$$\begin{aligned} Mac'(k, m) &= Mac(k, m) \\ Vrfy'(k, m, t) &= \begin{cases} Vrfy(k, m, t), & \text{dacă } m \neq 0^n \\ 1, & \text{altfel} \end{cases} \end{aligned}$$

Solution

MAC-ul nu este sigur pentru ca un adversar poate sa intoarca perechea validă $(0^n, 0^s)$.

Curs 13 – TLS

TLS - Transport Layer Security

► Este un protocol folosit de browser-ul web de fiecare data cand ne conectam la un browser folosind https ► primele versiuni se numeau SSL - Secure Sockets Layer - dezvoltat de Netscape (1995) - SSL 3.0, cea mai cunoscuta versiune ► TLS 1.0 apare in 1999, TLS 1.1 in 2006, TLS 1.2 in 2008 si versiunea actuala, sigura si eficienta TLS 1.3 in 2018 ► folosirea lui SSL 3.0, TLS 1.0 si TLS 1.1 trebuie evitata, toate trei prezinta probleme de securitate ► se recomanda folosirea minim a lui TLS 1.2

- Protocolul TLS permite unui client (de ex. browser web) și unui server (de ex. website) să se pună de acord asupra unui set de chei pe care să le folosească ulterior pentru comunicare criptată și autentificare
- Protocolul TLS constă din 2 părți:
 1. *protocolul handshake* - realizează schimbul de chei care stabilește un set de chei comune
 2. *protocolul record-layer* - folosește cheile stabilite pentru criptare/autentificare ulterioară
- In continuare vom prezenta protocolul handshake din versiunea curentă TLS 1.3

TLS 1.3 ► clientul si serverul, partajeaza, la finalul protocolului, cheile K_S si K_C , pe care le folosesc ulterior pentru criptare si autentificare ► in plus, clientul are garantia ca la final a partajat cheile cu server-ul legitim si ca acestea nu au fost interceptate sau modificate de o terta parte de ce? ► in handshake-ul din TLS 1.2, clientul si server-ul foloseau o schema de criptare cu cheie publica in locul schimbului de chei Diffie-Hellman ► clientul doar alegea o cheie K pe care o trimitea serverului criptata cu cheia lui publica

TLS 1.3

- aceasta a fost eliminată în TLS 1.3 pentru că nu asigură proprietatea de *forward secrecy* - care presupune că, în cazul compromiterii server-ului, cheile de sesiune anterioare nu sunt compromise
- în TLS 1.3, în ceea ce privește server-ul, cheia K este calculata pe baza secretului y in fiecare sesiune, secret care este unul nou de fiecare data si care nu trebuie mentinut intre sesiuni; deci compromiterea server-ului (aflarea cheii lui secrete) nu duce la aflarea cheii K
- în TLS 1.2, cheia secretă K ii este trimisa server-ului criptata cu cheia lui publica; compromiterea cheii secrete server-ului duce la compromiterea cheilor de sesiune din toate sesiunile
- în protocolul *record*, clientul și server-ul comunică în mod confidențial și autentificat folosind o schemă de criptare autentificată

TLS 1.3 vs. TLS 1.2

- număr redus de runde

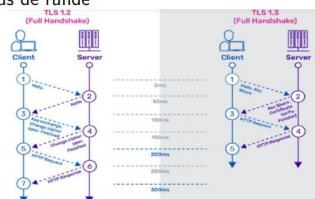


Figure: Sursa: www.a10networks.com

- eliminarea algoritmilor criptografici vulnerabili precum SHA-1, RC4, DES, 3DES, AES-CBC, MD5
- 0-RTT (zero round-trip) – reluarea unei sesiuni mai vechi cu un website e mult mai rapida